Effectiveness and Efficiency of Various Anti-unification Forms on Elixir Source Code

by

Christopher Kwiatkowski

CALGARY, ALBERTA

AUGUST, 2025

© Christopher Kwiatkowski 2025

Abstract

Source code generalization is fundamental to software engineering because it enables the discovery and extraction of recurring patterns and structures across codebases. Despite its importance in code analysis and transformation, there has been limited quantitative research comparing different generalization approaches, particularly regarding their effectiveness and computational efficiency.

This thesis investigates various forms of anti-unification — a formal problem involving generalization — for application to Elixir source code, specifically comparing first-order (FO) versus higher-order (HO) approaches, both with and without equational theory (ET). We develop a novel methodology to quantitatively evaluate these forms using effectiveness and efficiency measures, including non-variable count (NVC) and performance timing.

Our implementation encompasses source code encoding techniques, algorithm development incorporating the Needleman–Wunsch algorithm, and comprehensive measurement approaches. Through performance modelling, we establish that the NVC of the computed least general generalization (LGG) serves as a reliable predictor of execution time, although with moderate predictive power.

The research yields three key findings: (1) semantic generalization demonstrates substantially higher effectiveness than syntactic approaches, with FO+ET and HO+ET showing 590% and 703% effectiveness respectively compared to the FO baseline; (2) this increased effectiveness comes at a significant computational cost, with equational theory forms being approximately 1,000 times less efficient overall; and (3) the relationship between effectiveness gains and computational costs maintains similar proportions across different forms of generalization, supporting that the trade-off between FO and HO or with and without equational theory generalizes between different combinations.

These findings provide the first comprehensive empirical evaluation of effectiveness versus efficiency trade-offs in source code generalization, offering crucial insights for selecting appropriate generalization approaches based on specific requirements in software engineering applications.

Acknowledgements

All thanks go to God almighty for keeping me on this earth to complete the work needed.

Everything should be proven as true as possible, but not truer.

— Claude, 2025.

To my dearest friend Mac.

Table of Contents

Abstract ii

Acknowledgements iii

Epigraph iv

Dedication v

Table of Contents viii

List of Figures x

List of Tables xi

List of Abbreviations xii

1 Introduction 1
- 1.1 Generalization . . . 2
  - 1.1.1 Generalization in programming . . . 3
  - 1.1.2 Generalization quality . . . 4
  - 1.1.3 Generalization effectiveness . . . 5
  - 1.1.4 Generalization efficiency . . . 5
  - 1.1.5 Syntactic versus semantic . . . 6
- 1.2 Related work . . . 7
- 1.3 Thesis statement . . . 9
- 1.4 Thesis organization . . . 9

2 Background 10
- 2.1 Classical terms . . . 10
  - 2.1.1 Symbolic interpretation . . . 11
  - 2.1.2 Variable symbols . . . 12
  - 2.1.3 Substitution . . . 12
  - 2.1.4 Renaming . . . 13
- 2.2 Anti-unification . . . 13
  - 2.2.1 Least general generalization . . . 15
  - 2.2.2 First-order versus higher-order . . . 15
  - 2.2.3 Equational theory . . . 16
  - 2.2.4 Uniqueness . . . 17
- 2.3 Formal definitions . . . 17
  - 2.3.1 Terms . . . 17
  - 2.3.2 First-order anti-unification . . . 18
  - 2.3.3 Higher-order anti-unification . . . 19
  - 2.3.4 Equational theory . . . 19
  - 2.3.5 Anti-unification modulo equational theory . . . 19

2.4 Summary . . . 20

3 Approach **21**
3.1 Elixir source code . . . 21
3.1.1 Elixir syntax . . . 22
3.1.2 Term representation . . . 24
3.1.3 Compact quoted expression . . . 25
3.2 Anti-unification forms . . . 27
3.2.1 First-order and higher-order forms . . . 27
3.2.2 Equational theory . . . 28
3.2.3 Validation of implementation . . . 31
3.2.4 Combinations . . . 31
3.3 Comparing anti-unification forms . . . 31
3.3.1 Commonality counting . . . 32
3.3.2 Performance timing . . . 33
3.4 Summary . . . 33

4 Methodology **35**
4.1 Encoding source code as terms . . . 35
4.2 Anti-unification implementation . . . 38
4.2.1 First-order and higher-order . . . 38
4.2.2 Needleman–Wunsch alignment . . . 39
4.2.3 Example . . . 43
4.2.4 Equational theory . . . 49
4.3 Non-variable count implementation . . . 49
4.4 Performance measuring implementation . . . 50
4.4.1 Sampling . . . 50
4.4.2 Setup . . . 50
4.4.3 Measurement sample batching . . . 52
4.4.4 Memory access optimization . . . 53
4.5 Input data . . . 55
4.6 Summary . . . 55

5 Results and Analysis **56**
5.1 Performance data . . . 56
5.1.1 Analysis of block symbols . . . 58
5.1.2 Analysis of block arities . . . 59
5.2 Performance models . . . 61
5.2.1 Correlation coefficients . . . 61
5.2.2 First-order model . . . 61
5.2.3 Higher-order model . . . 64
5.2.4 First-order with equational theory . . . 64
5.2.5 Higher-order with equational theory . . . 64
5.3 Efficiency Comparisons . . . 64
5.3.1 First-order versus higher-order . . . 68
5.3.2 Without versus with equational theory . . . 69
5.4 Summary . . . 70

6 Discussion **71**
6.1 Effectiveness . . . 71
6.2 Efficiency . . . 72
6.2.1 Impact of equational theory on efficiency . . . 72
6.3 Trade-offs and generalizability . . . 74
6.4 Limitations . . . 74
6.4.1 Implementation limitations . . . 76

6.5 Threats to validity . . . 76
6.5.1 Construct validity . . . 76
6.5.2 Internal validity . . . 77
6.5.3 External validity . . . 77
6.6 Summary . . . 78

7 Conclusion **79**
7.1 Contributions . . . 80
7.2 Shortcomings of sequence alignment . . . 81
7.3 Future work . . . 81
7.3.1 Supporting more than two inputs . . . 81
7.3.2 Macro expansion . . . 82
7.3.3 Discovery of generalizations . . . 83

Bibliography **84**

List of Figures

2.1 Term represented as a labelled tree and vice versa. 11
2.2 Comparison of unification and anti-unification. Each σ (with or without a subscript) denotes a substitution being applied to the term at the tail of the arrow and producing the term at the head of the arrow. 14
2.3 Terms that produce $j(a, b)$ or $j(b, b)$ through substitution. The arrows indicate the direction of the substitution, though not the specific substitution, which can be inferred here by considering the difference from the source term to the target term for each arrow. 15

3.1 Generalized terms of two input terms $j(a, b)$ and $j(b, b)$ with their corresponding non-variable counts on the right. 32

4.1 Scoring matrix based on matches, mismatches, and gaps. 43
4.2 Initial empty scoring matrix. 43
4.3 Scoring matrix with first row filled in. 47
4.4 Scoring matrix with five cells filled in. 47
4.5 Scoring matrix with all cells filled in and the optimal path traced. 48
4.6 Line plot showing the change in median as more samples are taken. The experiment was repeated twice and so two lines are shown. 52

5.1 Scatter plot showing the log (base 10) of median timing across each of the four forms of anti-unification for each pair of terms. 58
5.2 For the FO form, using linear regression on the transformed data we have an R^2 value of 0.67 and it gives a final equation of $\log(y) = 0.70 \cdot \log(x) + 4.1$. 62
5.3 For the FO form, using linear regression on the untransformed data we have an R^2 value of 0.56 with an equation of $y = 70x - 150$. 62
5.4 For the HO form, using linear regression on the log transformed data we have an R^2 value of 0.63 and it gives a final equation of $\log(y) = 1.19 \cdot \log(x) + 3.3$. 63
5.5 For the HO form, using linear regression on the untransformed data we have an R^2 value of 0.50 with an equation of $y = 110x - 330$. 63
5.6 For the FO+ET form, using linear regression on the log transformed data we have an R^2 value of 0.82 and a corresponding equation of $\log(y) = 1.6 \cdot \log(x) + 7.8$. 65
5.7 For the FO+ET form, an R^2 value of 0.60 is obtained using the model $y = 2500x^{1.6}$. 65
5.8 For the HO+ET form, using linear regression on the log transformed data we have an R^2 value of 0.80 and a corresponding equation of $\log(y) = 1.8 \cdot \log(x) + 7.4$. 66
5.9 For the HO+ET form, an R^2 value of 0.64 is obtained using the model $y = 1600x^{1.8}$. 66
5.10 Histogram of NVC for the LGG of FO+ET and HO+ET when FO LGG NVC is zero. 67
5.11 Histogram of ratios of FO efficiency to HO efficiency. Approximately 45% of the data has an approximately equal ratio with FO efficiency being 1.0× the efficiency of HO. 68
5.12 Histogram of ratios of FO+ET efficiency to HO+ET efficiency. For approximately 45% of the data FO+ET efficiency is 1.25× the efficiency of HO+ET. 69
5.13 Histogram of ratios of FO efficiency to FO+ET efficiency. The median ratio puts FO efficiency at 400× that of FO+ET. 69

5.14 Histogram of ratios of HO efficiency to HO+ET efficiency. The median ratio puts HO efficiency at 500× that of HO+ET. 70

List of Tables

3.1 Equational theory for __block__ and nil. 28
3.2 Equational theory for `__block__` and `nil`. 29

4.1 Table of summary statistics for performance timings in nanoseconds of implementations with different memory related effects. 55
4.2 Table of summary statistics for the input dataset of terms. 55

5.1 Results for the anti-unification forms of first-order (FO), higher-order (HO), first-order with equational theory (FO+ET), and higher-order with equational theory (HO+ET). The execution times here are based on the median of 25 samples taken for each pair of terms. 57
5.2 Absolute totals of LGG NVCs and time costs for data with timings less than the 99.9th percentile. The time cost totals are based on the median times. Effectiveness is compared against FO as the baseline to give a percentage increase. 57
5.3 Minimum, median, maximum, and standard deviation (SD) statistics for the frequencies of the __block__ symbol in input terms and the anti-unifier terms produced by each form of anti-unification. 58
5.4 Frequencies for top ten occurrences of block arity frequencies for input terms being anti-unified. Under block arities the frequency of block arities in a term is given in a comma delimited list of arity-frequency colon separated pairs. The frequency column provides the number of terms that for the corresponding block arity frequency. 59
5.5 Frequencies for top ten occurrences of block arity frequencies for anti-unifier terms of various forms of anti-unification (first-order, higher-order, first-order with equational theory, higher-order with equational theory). Under block arities the frequency of block arities in a term is given in a comma delimited list of arity-frequency colon separated pairs. The frequency column provides the number of terms that for the corresponding block arity frequency. 60

List of Abbreviations

Abbreviation	**Definition**
API	application programming interface
AST	abstract syntax tree
AU	anti-unification
CPU	central processing unit
DSL	domain-specific language
ET	equational theory
FO	first-order [anti-unification]
FO+ET	first-order anti-unification modulo equational theory
HO	higher-order [anti-unification]
HO+ET	higher-order anti-unification modulo equational theory
LGG	least general generalization
NVC	non-variable count
PAM	Probabilistic API Miner

Chapter 1

Introduction

Source code serves as a foundation of software systems. As software engineering projects increase in size and complexity, the ability to understand, reuse, and maintain the code for these systems becomes increasingly difficult. One key aspect to handling this difficulty is in finding commonalities and differences between pieces of source code. Making these comparisons in source code is important because it allows for extraction of recurring patterns, structures, and functionalities across different portions of the codebase. Tools to aid with this process have potential to improve performance in software engineering tasks (Kim, 2007; Ain et al., 2019; Mahmoud et al., 2024).

A specific example is the problem of integrating disparate pieces of source code into a unified artifact. This can occur when forming generalized code templates from multiple concrete code examples. A more specific occurrence is when different sets of features are implemented at different times or by different programmers but later these are integrated into a single product.

For example, two companies may have different software systems with significant overlaps and significant differences. Consider the situation if those companies then merge into one: this creates a critical concern of how to continue to develop the software systems. Should one system be chosen in favour of the other and migrated to or should both systems be maintained independently? Does this decision even need to be made at such a coarse-grained level of the whole system or can the systems be decomposed into smaller pieces?

Research in formal models of generalization can provide knowledge to help in this process (Krumnack et al., 2007). In particular, source code from two sources can be compared to find commonalities and differences while also forming a generalization of the two (Mahmoud et al., 2024). However, the effectiveness of this depends on incorporating knowledge of what the source code does (i.e., its semantics). Without the ability to compare the semantics of the source code the comparison will only find commonalities and

differences in how the source code looks in terms of keywords, identifiers, statements, etc. (i.e., its syntax). But, while syntactic comparison can be done very efficiently, semantic comparison is much more expensive (Gabel et al., 2008). This raises the question of when to choose syntactic generalization over semantic generalization.

To help address the problem of choosing between different forms of generalization, this thesis quantifies and compares the trade-offs in efficiency versus effectiveness for a few formal models of syntactic and semantic generalization. For the sake of keeping a manageable scope, additional models that incorporate even more semantic knowledge are treated as out of scope and left for future research work.

1.1 Generalization

A generalization represents the commonalities between the items being generalized while abstracting away the differences.

The process of generalization (or simply, generalization — the context typically disambiguates from the generalization-as-result) is simple to explain conceptually but its formalization is challenging. The rough idea is that, given some starting elements, we can form a generalization of these by applying abstraction to each element until we obtain a common generalized element, which is the commonality shared between them, abstractly, concretely, or a mixture of the two. For example, given the elements "19" and "23" we can form the generalization "prime" since "19" is a prime number and "23" also is a prime number.

The ubiquity of this process in human thought makes our own experience of forming generalized concepts by abstracting concrete instances feel intuitive. However, *articulating* the precise steps of the process is difficult. In particular, there are many different valid abstractions that we can consider. For example, both "19" and "23" are also odd numbers so we could generalize these as "odd" instead of "prime." Further possibilities include "two-digit number", "21 ± 2", as well as just "element."

Deciding which abstraction possibilities to consider and which should be ignored is a key step when rigorously defining a generalization process. Depending on the restrictions (or lack thereof) on these abstractions, a process of generalization will have varying complexity. In theoretical terms, this includes the problem of decidability and of infinite versus finite sets of solutions (Bürckert et al., 1989; Alpuente et al., 2014). Results for these different forms of generalization have been extensively studied under the formal theory called *anti-unification* (Bürckert et al., 1989; Krumnack et al., 2007; Bulychev et al., 2010; Baumgartner et al., 2017). We study the formal problem of anti-unification (AU) and its variations in a later chapter; the formal problem is used in this thesis to provide a solid foundation for formalizing generalization.

While formal models allow reasoning about generalization in the abstract, we also wish to apply such

a model to a real-world context. For this thesis the context is generalization of source code written in a programming language.

1.1.1 Generalization in programming

To apply generalization in programming, a programming language must support abstraction. Different capabilities can enable abstraction; however, procedure definitions (or a related notion such as method or function definitions) are considered fundamental.

Most programming languages include this capability of defining procedures in a manner that separates the definition from the procedure's invocation. From this separation, there arise two abstraction mechanisms (Liskov and Guttag, 2000): (1) *abstraction by parameterization*, and (2) *abstraction by specification*. Liskov and Guttag describe these as follows:

> *Abstraction by parameterization* abstracts from the identity of the data by replacing them with parameters.
>
> *Abstraction by specification* abstracts from the implementation details (how the module is implemented) to the behavio[u]r users can depend on (what the module does).

To illustrate the application of these abstraction mechanisms, a refactoring is shown between Listing 1.1 and Listing 1.2. Here an abstraction by parameterization is applied to the concrete data "A dog" and "A cat" to parameterize them in the function printBoth as parameters named a and b. Simultaneously, abstraction by specification is applied to the concrete print statements replacing them with an invocation to the function named printBoth.

```
1 print("A dog");
2 print("A cat");
```

Listing 1.1: Initial code.

```
1 function printBoth(a, b) {
2     print(a);
3     print(b);
4 }
5 printBoth("A dog", "A cat");
```

Listing 1.2: Refactoring applied.

In this example, a particular programming language is not assumed. Depending on the actual programming language there will be variations in what abstraction mechanisms are available along with variations in their syntax. However, in principle we might have other abstractions such as one that allows further generalization of the function to support not only two arguments but an arbitrary number of arguments. Another case could involve parameterization of only the differences between the data while keeping the common prefix `"A "` hard-coded within the function definition. Given these various possible abstractions, the difficult question facing a programmer is which one to pick. This is the question of *quality* of generalization.

1.1.2 Generalization quality

Like the concept of generalization, the concept of quality is simple but its precise description tends to be elusive (Miles et al., 1995). Conceptually, quality is the determination by some user of which product they have a preference for in terms of its value to them.[1] So by intuition, generalizing the elements "19" and "23" as "prime" seems better than "odd" or just "element," but without articulating the process for determining quality there is no possibility to reason formally about it or to implement it in an executable program.

A partial solution is the idea of a *least general generalization* (LGG) when different generalizations can be more general or less general than each other. Continuing with the example of "19" and "23" and assuming that every possible element is an actual element, the generalization "element" is the most general while "prime" gives further specifics. Consequently we can say that "element" is a generalization of lower quality compared to "prime" because it lacks specifics. In contrast, comparing "prime" and "odd" cannot be done in this manner since not all prime numbers are odd and not all odd numbers are prime. In such a case, we must admit that in general there is no unique answer to which generalization is of the best quality.

Another partial solution is the idea of a proxy measure such as similarity between the generalized element and the starting inputs. A measure of similarity is treated as a proxy since quality cannot be reduced to a purely objectively measurable quantity due to the dependency upon someone actually using what is produced. Generally, it should not be assumed that the quality determined by a user has a perfect alignment with a particular similarity measure. However, proxy measures such as similarity have been found useful as part of heuristic approaches to finding high quality generalizations (Cottrell et al., 2008).

The approach in this thesis uses these two ideas, but instead of answering the question of quality, we tackle the question of effectiveness instead.

[1] This conceptualization may seem to imply quality is entirely subjective; however, objective factors may still be a significant part of the user's determination and what they value.

1.1.3 Generalization effectiveness

A process of generalization does not have inherent characteristics that create quality in its results. Instead quality emerges when the process is correctly fit to the determination of quality made by the user of the results. Therefore, making "improvements" to the process has no meaningful relationship to quality without an inclusion of the user that brings meaning to the concept of quality.

At first impression, it seems further progress cannot be made without a method of determining quality. However, instead of treating quality as the ultimate goal, we can treat adaptability to different quality needs as our objective in studying different processes of generalization. Essentially we can prepare ahead of time with a progression of processes to use when given a specific determination of quality from the user.

By analogy, a workshop has many tools that might typically be organized by their effectiveness for different tasks (e.g., drill bits of different sizes). Producing quality occurs once a specific need is known and then the appropriate tools are selected. They cannot be organized according to "the quality they produce" since this relationship does not exist until someone with a real need shows up.

In the context of generalization, the scale of how many abstraction possibilities are considered affects the effectiveness of the generalization process. The more abstraction possibilities are considered, the more generalizations can be produced. Of course this depends on the particular inputs being generalized, but in general we can expect that more results (generalizations) should be produced when a larger range of possibilities is searched. Note that this range could be infinite and so this concept of effectiveness becomes much more complex to formalize than if this range were finite.

Later in this thesis, a formalized definition of effectiveness for generalization is given, but here we introduce just the concept. For the purposes of this thesis, effectiveness is considered to be the degree to which a process produces useful results. This allows us to compare different processes to understand their relative effectiveness without depending on a fixed definition of quality. In particular, it is the relationship between different processes in terms of their effectiveness and efficiency that we are interested in studying. The concept of efficiency is introduced in the next subsection.

1.1.4 Generalization efficiency

Given different generalization processes to choose from, we obviously would like to pick the most effective one; however, two problems arise. Firstly, the notion of "the most effective" needs to be defined and such an instance does not necessarily need to exist. Secondly and more practically, we need to consider the cost associated with the effectiveness.

When effectiveness is normalized by its cost we derive the concept of efficiency. For the purposes of

this thesis, efficiency of generalization is its effectiveness scaled by its cost. Typically, computation cost is measured in terms of time and in this thesis this is the kind of efficiency considered, although other costs could be considered also. A formalization of efficiency is provided later.

It is important to note that efficiency is a separate concern from effectiveness. If a process is not effective then improvements in efficiency provide no value. However, an inefficient process can still be effective and provide value. By first identifying the level of effectiveness required, we can know where to focus efforts on improving efficiency.

By studying both effectiveness of different generalization processes along with their efficiencies, we can better predict the outcome of choosing one process over another. One key aspect to understand is whether a trade-off needs to be made between effectiveness and efficiency. If a trade-off occurs then there is value to having different processes available with different trade-offs. This allows the accommodation of different real-world situations where the value of results and their production costs relates to external factors.

1.1.5 Syntactic versus semantic

The range of different generalization processes can be divided in many different ways; however, a general distinction is typically made between syntactic and semantic generalization. Assuming our input elements are decomposable into symbols (details on this are discussed in a later chapter), syntactic generalization involves a comparison of just raw symbols while semantic generalization deals with comparing the interpretation of the symbols.

To illustrate the difference between syntactic and semantic comparison we can take the generalization of two symbols "23" and "93" as an example. At the level of comparing symbols the only information available is that these two are not literally identical. However, if we interpret these as decimal numerals then each of these are composed of two symbols (their digits) "2, 3" and "9, 3", respectively. Now when we compare these we can find the commonality of ending in the digit "3." This allows us to form the generalization "X, 3" where "X" represents an arbitrary digit.

In the context of a programming language we may have a function "digits" which returns the digits in a particular base. For example, the expression "digits(93, 10)" would evaluate to "9, 3" and "digits(23, 2)" to "1, 0, 1, 1, 1." Now we can consider comparing the expressions "digits(23, 2)" and "digits(93, 3)" syntactically versus semantically. A syntactic comparison will not consider the meaning of the "digits" function and so the best generalization we can offer is "digits(X, Y)" where "X" and "Y" are our placeholders (a precise formulation of this is provided later in the thesis). A semantic comparison on the other hand allows interpreting "digits" to obtain the meaning of the expressions as "1, 0, 1, 1, 1" and "1, 0, 1, 1, 0." This

allows us to form a generalization that preserves more commonalities in the generalized expression "1, 0, 1, 1, X."

The key difference between syntactic and semantic generalization is that syntactic generalization uses only a "literally identical" comparison while the semantic generalization generalizes this to a "semantically equivalent" comparison. For this reason comparing "23" and "93" syntactically can typically be mapped to a single integer comparison in hardware. However, a semantic comparison becomes significantly more complex as it can involve comparing many representations because of semantic equivalences between them. Suppose the function "undigits" constructs a number from some given digits and base, then we will have equations such as $93 = \text{undigits}([1, 0, 1, 1, 0], 3)$ and $23 = \text{undigits}([1, 0, 1, 1, 1], 2)$ in addition to many *many* more.

The addition of such equations in semantic generalization has a precise formulation in anti-unification theory. This form of anti-unification with particular equations forming a concrete "equational theory" is known as anti-unification *modulo equational theory* and the details of this formalization will be discussed later in this thesis. For now the key point is that semantic generalization offers a potentially more effective process of generalization in terms of how many commonalities occur in the generalizations produced. However, the additional overhead due to the complexity of semantic comparison means the efficiency in terms of computational cost could be problematic.

In this thesis we are interested in studying this trade-off between effectiveness and efficiency for syntactic versus semantic generalization. We accomplish this by defining measures for effectiveness and efficiency, selecting various forms of anti-unification to study (and defining an equational theory of interest), then obtaining results for our implemented forms using a dataset of real-world source code.

1.2 Related work

In a later chapter, we analyze the literature to consider where generalization is applied along with its strengths and weaknesses, including work both involving and not involving anti-unification. Here, we give a short overview relative to applied work on anti-unification, and only briefly mention one work on generalization not involving anti-unification to help motivate the path ahead.

Various research has previously considered generalization of source code based on anti-unification. This includes code clone detection (Bulychev, 2009; Li and Thompson, 2010), source code reuse (Cottrell, 2008), application programming interface (API) usage analysis (Zirakchianzadeh, 2016), and API usage template extraction (Mahmoud et al., 2024). The variety of applications speaks to the prevalence of generalization problems across many software engineering situations.

These works are explicit about their use of anti-unification as a theoretical basis for generalization as they

recognize generalization as a fundamental concern embedded in the specific software engineering contexts. The advantage of this is a *precise* definition of what problem is being solved, thereby allowing an evaluation of the correctness of implementation. Without such a definition, evaluating correctness of a tool lacks a precise meaning and, instead, the tool needs to be treated as giving an approximate solution to a *vague* notion of a problem. Although such an approach does occur in other research and might still lead to useful tools (perhaps validated through human trials), the main criticism of this approach is a lack of rigour in planning further development.

As an example, there are tools for extracting generalized patterns that do not make explicit use of anti-unification. One such tool is PAM (Probabilistic API Miner) tool which uses probabilistic modelling of API call sequences to extract sequence patterns from source code (Fowkes and Sutton, 2016). However, the statistics theory behind this tool is not actually "competing" with anti-unification as the probabilistic model addresses the concern of evaluating the "interestingness" quality of patterns. The limitation of the approach in PAM that anti-unification addresses is the generalization of the patterns produced to arbitrary structures beyond just sequences. Another key limitation that anti-unification can address is the consideration of semantic equivalences between expressions in the source code. These additional concerns present opportunities to further develop and generalize the approach used by PAM to solve harder problems. Doing so requires a generalized formulation of generalization (e.g., to include tree structures and semantic comparisons) which is precisely what existing anti-unification research can already provide.

As a generalized formulation of generalization, anti-unification provides insights into the fundamental trade-offs between the *different problems* and gives a clearer path for advancement. This multiplicity of problems under the umbrella term "generalization" is a key insight of anti-unification theory and explains that generalization cannot be understood as a single problem solvable by a single universal tool. Instead, different forms of anti-unification will offer different trade-offs between traits such as decidability, effectiveness, and efficiency. There is no absolute answer to which form is the "best" because the choice is dependent on the usage scenario.

Unfortunately, an empirical comparison between forms of anti-unification applied to source code is absent in the existing literature. In particular the advantages and disadvantages between syntactic and semantic generalization are not well described. In this thesis we aim to close this gap by performing a direct comparison between a specific selection of forms of anti-unification.

1.3 Thesis statement

The thesis of this dissertation is that: (a) semantic generalization is more effective than syntactic generalization; (b) semantic generalization is less efficient than syntactic generalization; and, (c) the quantified trade-off between effectiveness and efficiency has some generalizability between different forms.

1.4 Thesis organization

The thesis is organized as follows. Chapter 2 covers the background on terms and substitution in the context of various forms of anti-unification including first-order versus higher-order and syntactic versus with equational theory. Chapter 3 describes the approach to implementing and performing a comparison of the four forms of anti-unification studied in this thesis. Chapter 4 discusses the details of implementation. Chapter 5 presents the results along with some analysis. Chapter 6 discusses the results and analysis, threats to validity, and limitations. We conclude in Chapter 7 with contributions and future work.

Chapter 2

Background

This chapter introduces ideas surrounding formal problems (collectively known as anti-unification) regarding generalization, including terms, substitution, and least general generalization. This provides the background needed to understand our approach to generalization of a term representation of source code using the four forms of anti-unification we compare in this thesis: first-order versus higher-order and syntactic versus modulo equational theory.

In Section 2.1, we explain terms, variable symbols, substitution, and renaming substitutions. Then we discuss anti-unification in Section 2.2, covering least general generalization, the difference between first-order versus higher order forms, the form with equational theory added, and the property of uniqueness for specific forms.

2.1 Classical terms

Terms are fundamental elements that can represent objects, entities, properties, and relationships. A key aspect of terms is that they can be composed into new terms by relating one or more subterms with a symbol denoting the kind of structure formed in the composition. For example, a sequence structure can be denoted by a "cons" function that links one element to another sequence or to a special "nil" symbol representing the end of a sequence. This could be represented in syntactically different ways such as "cons(1, cons(2, nil))", "[1 | [2 | [ ]]]", or "[1, 2]", but the meaning (i.e., its semantics) is still the same.

The structure of terms is more generally the same notion as that of a tree from mathematics. Formally, a term corresponds with a finite ordered tree with each leaf node labelled by either a *constant* or a *variable* (Dershowitz and Jouannaud, 1990). Internal (non-leaf) nodes are labelled by function symbols with a positive arity equal to their count of immediate child nodes. Constants are considered function symbols of arity zero,

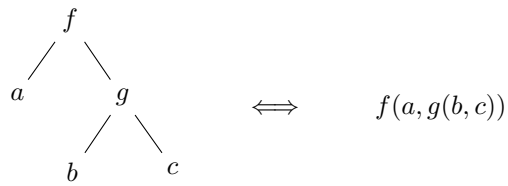

Figure 2.1: Term represented as a labelled tree and vice versa.

necessarily making them leaf nodes since they have no children.

The correspondence between a term and a tree is illustrated in Figure 2.1. The term $f(a, g(b, c))$ involves two function symbols f and g and three constants a, b, and c which correspond with the non-leaf node labels and leaf node labels, respectively, in the tree representation. To convert from the term to the tree, we maintain the notion of the current position in the tree so far, creating a node when we encounter a constant or variable, adding an edge and descending to the next level when we encounter an open parenthesis, adding an edge from the same parent to the current level when we encounter a comma, and returning to the level above when we encounter a close parenthesis; the final position will be the root of the tree. To convert from the tree to the term, we perform an in-order traversal of the tree, recording the node label when we encounter a node, an open parenthesis when we descend a level, a comma when we encounter a sibling on the current level, and a close parenthesis when we return to the previous level.

The "cons" function from the earlier example is a non-leaf node with arity two and "1", "2", and "nil" are constants with arity zero. Conventionally, the other notations such as "[ ]" would also be treated as symbols; however, this derives from a purely computational perspective based on symbolic interpretation.

2.1.1 Symbolic interpretation

The notation used in source code is conventionally interpreted as exclusively symbolic since "computation is defined in terms of symbol manipulation" (Searle, 2002). When we are formalizing the interpretation of source code by computing machines, it is reasonable to model source code as comprised exclusively of symbols. But people, in contrast to machines, perceive source code text in a broader perspective that is not restricted to just symbols. The notation "[ ]" in particular is more precisely an *iconic* representation rather than a *symbolic* one according to Peirce's triadic model of the sign (Peirce, 1902). This is because a symbol has an arbitrary relation to the object it signifies, but to a person the notation "[ ]" is not entirely arbitrary. So, for a computer compiling source code, the notation "][" to signify "empty" would be no less reasonable. For a person, however, the notation "[ ]" is more reasonable because it visually conveys the meaning "empty" and therefore it is an iconic representation.

This distinction is important because we must be clear, when formalizing terms and the generalization of terms, that our formal model assumes a purely symbolic interpretation — terms are just symbolic representations. When formal definitions are given in Chapter 3, the definitions also assume that the symbols needed to construct terms are already given. Similarly, there is an assumption that symbols are divided into *variable* symbols and *non-variable* symbols (function symbols and constant symbols) and that this division is also already given.

2.1.2 Variable symbols

Generalization depends on abstraction, and the mechanism for abstraction of terms comes from the introduction of *variable* symbols. This raises an important question of where variable symbols come from. To answer this, we must refer to the definition of a symbol.

As already mentioned, according to Searle (2002), "computation is defined in terms of symbol manipulation" but also "there are no purely physical properties that zeros and ones or symbols in general have that determine that they are symbols." He explains further that "something is a symbol only relative to some observer, user or agent who assigns a symbolic interpretation to it."

In line with this reasoning, we can argue that distinguishing between variable symbols and non-variable symbols must be relative to some interpretation. That is, the theory of terms does not prescribe a method of determining whether a symbol is a variable or not; however, the theory does formalize the meaning of the distinction. In the theory of terms, the notion of (variable) *substitution* gives special significance to variable symbols relative to non-variable symbols. Based on what substitutions belong in our formal model, we can determine the appropriate separation of symbols into variable and non-variable.

2.1.3 Substitution

In the context of terms, "a *substitution* is a special kind of replacement operation, uniquely defined by a mapping from variables to terms" (Dershowitz and Jouannaud, 1990). For the purposes of formal reasoning, we can formalize a substitution as a mathematical function with a domain of the set of variables; however, this elides an important characteristic of a substitution.

An appropriate conceptualization of substitution requires that a substitution imply *some* variation. The importance of this detail is discussed by Geach (1951) and by Dilworth (1988) who argue in the context of the concept of identity that substitution cannot meaningfully involve a thing being substituted with the same thing. This perspective is also expressed by Lassez et al. (1988) in describing substitution as "destructive assignment" that "assigns terms to variables."

Because a variable cannot be meaningfully said to be substituted by itself, care needs to be taken in the representation of a substitution. We can use set notation to represent a substitution in the manner of "$\{x_i \to t_i : x_i \neq t_i\}$ where x_i is an element of the set of variable symbols and t_i is an element of the set of (all) terms. This ensures that the representation uniquely defines a substitution. Otherwise without the restriction, the sets $\{X \to a, Y \to Y\}$ and $\{X \to a\}$ (with uppercase letters denoting variables) would both be valid representations of the same substitution. Since the sets are not equal, the representation would not be unique. With the restriction in place, uniqueness of representation is easier to ensure for substitutions.

2.1.4 Renaming

Special cases of substitution arise when variables are substituted only by variables (but different from themselves). If additionally the corresponding mapping from the set of all variables $\mathbb{X}$ to the set of all variables $\mathbb{X}$ is a bijective mapping then by implication the substitution is invertible. This kind of substitution is called a *permutation of variables* or a *renaming of variables* (Palamidessi, 1990).

The significance of a *renaming* is that when we compose the application of a renaming substitution with its inverse the result is equivalent to an identity function. Palamidessi treats this as an "empty substitution", meaning that there is no substitution occurring upon any variable and so applying it is identical to nothing occurring. In the context of a physical cost being imposed on performing substitutions, such situations would constitute optimization opportunities.

One case of substitution that resembles a renaming but is not, by our definition, is where a variable is substituted with another variable that is not itself substituted. For example, the term $f(A, B)$ could have a substitution $\{A \to B\}$ applied to it resulting in the term $f(B, B)$. Because applying the inverse substitution $\{B \to A\}$ to $f(B, B)$ results in $f(A, A)$, we see that composing these would not be equivalent to an empty substitution; thus, this example of a substitution is not a renaming.

2.2 Anti-unification

With terms and substitution introduced, the concept of generalization of terms can be explained.

Generalization is described by the formal problem of *anti-unification*. The first formulation of an algorithm based on anti-unification is credited to a paper by Plotkin (1970) on inductive generalization. Around this time, it was recognized that this approach to generalization corresponds with the dual of *unification* and so the name *anti*-unification was adopted. For this reason, Plotkin actually makes no explicit reference to "anti-unification", but the concept is the same.

Both unification and anti-unification use the notions of substitutions and terms to rigorously define

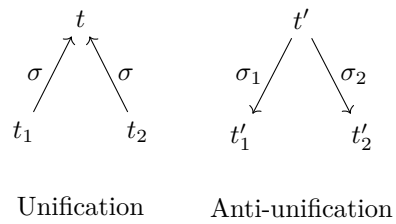

Figure 2.2: Comparison of unification and anti-unification. Each σ (with or without a subscript) denotes a substitution being applied to the term at the tail of the arrow and producing the term at the head of the arrow.

problems involving finding substitutions and terms (typically denoted by σ or σ_i and t or t_i, respectively) given some initial terms.

The general relationship between unification and anti-unification is shown in Figure 2.2. Unification is depicted as relating two terms t_1 and t_2 to the (unified) term t by the same substitution σ. Anti-unification on the other hand is shown as relating the (anti-unified) term t' to two terms t'_1 and t'_2 by two substitutions that do not need to be the same. Note that in the unification case, t will be more *specific* than t_1 and t_2 due to the direction of substitution application (except in the case of $t_1 = t_2$); this means that t will be more concrete (less abstract) as it contains fewer variables. For anti-unification the inverse is true, with t' being more *general* (i.e., more abstract) than t'_1 and t'_2 (except in the case of $t'_1 = t'_2$).

Bürckert et al. (1989) describe unification theory as:

> *[...] concerned with problems of the following kind: given two terms built from function symbols, constants and variables, do there exist terms that can be substituted for the variables such that the two terms thus obtained become equal?*

Thus, anti-unification can be described as problems of the following kind: given two terms, does there exist a single generalized term containing variables that can be substituted in at least two ways such that each of the given two terms can be obtained?

More formally, given terms t'_1 and t'_2, the anti-unification problem is that of finding a generalized term t' along with substitutions σ_1, σ_2 that satisfy $\sigma_i t' = t'_i$ for $i \in \{1, 2\}$ where $\sigma_i t'$ denotes applying the substitution σ_i to the term t'. Note that the generalized term is also known as an *anti-unifier*.

However, this description is not sufficiently interesting because a trivial solution always exists: given two terms t_1 and t_2, there is always a generalized term of a single (fresh) variable X with substitutions $\{X \to t_1\}$ and $\{X \to t_2\}$ that satisfies the anti-unification problem.

This trivial solution, however, is too general to be useful in most cases. Instead we consider the problem of finding a *least general generalization* (LGG) to be more interesting, which involves finding a less abstract

solution, i.e., one that maintains more information from the original two terms.

2.2.1 Least general generalization

We start from an intuitive explanation as to the nature of a least general generalization. To illustrate the relationship between generalized terms, Figure 2.3 shows an LGG $j(X, b)$ (roughly in the centre of the figure) formed from two given terms $j(a, b)$ and $j(b, b)$ (at the bottom). Here we denote variables by uppercase letters X and Y, j denotes a function symbol of arity two, and a and b are constants. As we follow the arrows further back towards the variable X at the top we see that the term becomes less specific. For example, we observe that, when b is replaced by Y in the abstraction of $j(X, b)$ to $j(X, Y)$, the information about the second argument to j is lost. In this diagram, the generalized term that preserves the most information is $j(X, b)$ corresponding with the least general generalization. A formal definition of LGGs, given the input terms being generalized, is presented in Chapter 3.

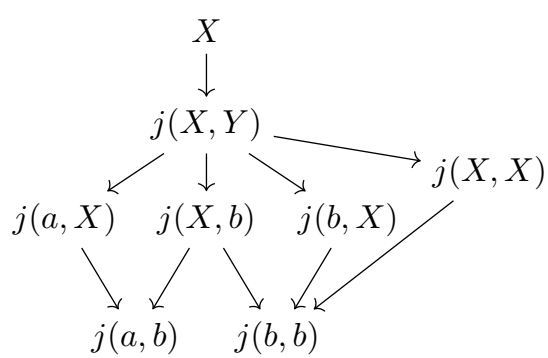

Figure 2.3: Terms that produce $j(a, b)$ or $j(b, b)$ through substitution. The arrows indicate the direction of the substitution, though not the specific substitution, which can be inferred here by considering the difference from the source term to the target term for each arrow.

2.2.2 First-order versus higher-order

The classical terms described in this chapter are considered "first-order" terms because variables, like constants, cannot have subterms. Consequently a "term" such as $X(a, b)$, where X is a variable, is not a valid first-order term. Extending the definition of terms to allow this kind of expression leads to a definition of higher-order terms.

In the context of anti-unification, this means that, with the first-order restriction, the LGG for terms $f(a)$ and $g(a)$ is necessarily X. If, in the case of a higher-order form, we were to allow variables to replace function symbols of non-zero arity then $X(a)$ would be a legal generalization here. Compared to the term X, the higher-order term $X(a)$ is less general and consequently captures more information about commonalities

between the terms being generalized. Applying our definition of LGG again, we observe that X is not a (higher-order) LGG here because there is no substitution possible for $X(a)$ that produces X.

2.2.3 Equational theory

The forms of anti-unification discussed so far have assumed that equality of terms means syntactic equality. That is, two terms are equal if and only if they have an identical representation. Anti-unification based on syntactic equality is also known as *syntactic generalization*.

When semantic equivalence is of interest, we need to introduce an *equational theory* that defines equalities between terms with differing syntactic representations. This is critical if we are interested in generalizing entities that have semantics different from those of trees. When the semantics are considered in the generalization, we consider this to be *semantic generalization*.

Given a finite or infinite set of equations E, where an equation is formalized as an unordered pair of terms $\{s, t\}$, the equational theory of E is a set of equations that includes all those inferred from the equations in E as axioms (Dershowitz and Jouannaud, 1990).

As a concrete example, consider a sequence of expressions grouped into a *block expression*. This is a typical construct supported by programming languages enabling multiple expressions or statements to be treated as a single "body" unit such as the body of a function definition or of a method declaration. A sequence represented as $s(b)$ is a subsequence of a sequence represented as $s(a, b)$. However, under syntactic equality these terms are different and their LGG would be simply X. But we wish to be able to form the generalization $s(X, b)$ in order to capture the commonalities arising from the semantics of the sequence structure. Formally, this is not immediate because the two function symbols are of differing arity.

One approach to model this situation would be to introduce an equational theory so that a sequence can have "empty" (neutral) elements interspersed while retaining semantic equivalence; for example, a "no-op" is a common feature in low level programming languages that results in no change to a computation, and we can represent this as nil for the sequence functions. Then, we may have an equation $s(b) = s(nil, b)$ and the generalization $s(X, b)$ can be formed from $s(nil, b)$ and $s(a, b)$. Applying the substitution $\{X \to nil\}$ to $s(X, b)$ produces $s(nil, b)$ which, although not syntactically equal to $s(b)$, is semantically equivalent due to the equation we defined. (To be rigorous, care must be taken to differentiate such neutral elements on the basis of the arity of the variable for which they may substitute; in practical situations, this detail is implied and ignoring it causes no difficulties.)

This kind of problem where we wish to find substitutions which produce terms that are equal under the equational theory (but not necessarily syntactically equal) is called *anti-unification modulo equational theory*

(or more simply *E-generalization*).

An important characteristic of these anti-unification modulo equational theory problems is that, depending on the equational theory, there may be more than one unique LGG solution and the decidability of reaching any solution may not be guaranteed (Bürckert et al., 1989).

2.2.4 Uniqueness

The significance of the potential non-uniqueness of solutions in anti-unification modulo equational theory is related to the fact that syntactic anti-unification *does* have a unique LGG modulo *renaming* (as defined earlier). This means that, for any two LGGs found as solutions to a syntactic anti-unification problem, they must either be syntactically equal or have a renaming substitution that form an invertible mapping between them.

2.3 Formal definitions

The formalization of generalization requires a formal definition of terms (the objects being generalized) and a formal definition of the generalization problem. When the terms involved are first-order terms, the form of anti-unification is first-order; the situation is equivalent for higher-order. We start with defining first-order then proceed to higher-order. Then we introduce the form that considers semantic equivalences — anti-unification modulo equational theory, also called E-generalization.

2.3.1 Terms

The standard definition of terms in first-order logic, as given by Dershowitz and Jouannaud (1990), defines the set of terms $\mathbb{T}(\mathbb{F}, \mathbb{X})$ over a signature $\mathbb{F}$ of function symbols and a set $\mathbb{X}$ of variables as the smallest set containing $\mathbb{X}$ such that $f(t_1, \ldots, t_n) \in \mathbb{T}(\mathbb{F}, \mathbb{X})$ whenever $f \in \mathbb{F}$ has arity n and $t_i \in \mathbb{T}(\mathbb{F}, \mathbb{X})$ for each $i = 1, \ldots, n$.

While this definition suffices for many applications, our work requires a more general framework that can accommodate variable function symbols. We therefore adopt the following extended definition:

Definition 1 (Terms)**.** *Let $\mathbb{F}$ be a set of function symbols, $\mathbb{X}$ a set of individual variables, and $\mathbb{Y}$ a set of function variables. The set of* terms $\mathbb{T}(\mathbb{F}, \mathbb{X}, \mathbb{Y})$ *is defined inductively as:*

- *Every individual variable $x \in \mathbb{X}$ is a term.*
- *If $f \in \mathbb{F}$ has arity n and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term for $n \geq 0$.*

- *If $Y \in \mathbb{Y}$ has arity n and $t_1, \ldots, t_n$ are terms, then $Y(t_1, \ldots, t_n)$ is a term for $n \geq 0$.*

This generalized notion of terms proves essential for our analysis of functional programming constructs, where functions themselves may serve as parameters or return values. The distinction between different classes of terms naturally emerges from this framework.

Definition 2 (First-order terms)**.** *A term $t \in \mathbb{T}(\mathbb{F}, \mathbb{X}, \mathbb{Y})$ is called* first-order *if it contains no function variables from $\mathbb{Y}$. Equivalently, t is first-order if $t \in \mathbb{T}(\mathbb{F}, \mathbb{X}, \emptyset)$.*

Definition 3 (Higher-order terms)**.** *A term $t \in \mathbb{T}(\mathbb{F}, \mathbb{X}, \mathbb{Y})$ is called* higher-order *if it contains at least one function variable from $\mathbb{Y}$.*

This terminology aligns with the conventional understanding that higher-order structures involve abstraction over functions rather than merely values. In the context of source code analysis, higher-order terms enable us to capture patterns where the computational behaviour itself varies, not just the data being processed.

2.3.2 First-order anti-unification

Based on the first-order terms, the problem of finding generalizations for these is defined by first-order anti-unification. Formally, given first-order terms t_1 and t_2, the (first-order) anti-unification problem is that of finding a generalized term t along with substitutions σ_1 and σ_2 that satisfy $\sigma_i t = t_i$ for $i \in \{1, 2\}$ where $\sigma_i t$ denotes applying a substitution to a term. Substitutions are represented using set notation in the manner "$\{x_i \to t_i : x_i \neq t_i\}$ where $x_i \in \mathbb{X}$ and $t_i \in \mathbb{T}(\mathbb{F}, \mathbb{X})$." It is important to note that each replacement $x_i \to t_i$ in a substitution σ takes place simultaneously with respect to the others in σ. We denote applying substitution to a term by prefixing the term with the substitution, as in σt.

Although there may be multiple generalizations possible, we are specifically interested in finding a least general generalization (LGG). A formal definition of LGG uses substitution as follows.

Definition 4 (Least General Generalization of First-Order Terms)**.** *Given first-order terms t_1 and t_2, their* least general generalization *(LGG) is a generalized term t_{lgg} such that $\sigma_i t_{lgg} = t_i$ and, for any possible generalized term t' such that $\sigma_i t' = t_i$, there exists a substitution σ such that $\sigma t' = t_{lgg}$.*

This definition refers to <u>*an*</u> LGG rather than <u>*the*</u> LGG since a variable renaming substitution could possibly be applied to produce a different term that is also an LGG. But if all of these LGGs are renamings of each other, then treating these as a single equivalence class leads to a notion of *the* LGG that is "unique up to renaming" (Plotkin, 1970). This becomes relevant later in this chapter when anti-unification modulo equational theory (which does not generally guarantee a unique LGG) is discussed.

2.3.3 Higher-order anti-unification

The higher-order form in contrast to first-order includes function terms where function symbols can be generalized by variables. For example, given the two terms $f(a)$ and $g(a)$: the first-order anti-unification form results in an LGG X, while the higher-order form results in the LGG $Y(a)$ where X and Y are variables.

2.3.4 Equational theory

The addition of an equational theory permits equating terms that differ syntactically but should be considered semantically equivalent. We begin with formal definitions of equations, equational theory, and E-equality. Then we replace the syntactic equality in the previous anti-unification definitions to obtain the formal definitions for the problems of (1) first-order anti-unification modulo equational theory (FO+ET) and (2) higher-order anti-unification modulo equational theory (HO+ET)

Definition 5 (Equation (Bürckert et al., 1987))**.** *An* equation *is a pair of terms usually denoted by $s = t$. Given a set of equations E and a single equation $s = t$, we write $E \vDash s = t$ to denote that $s = t$ is true in every model of E ($s = t$ is a consequence of E).*

Definition 6 (Equational Theory (Bürckert et al., 1987))**.** *An* equational theory *is a set of equations T with $T \vDash s = t \iff s = t \in T$, i.e., T consists exactly of its consequences. The least equational theory $T(E)$ that contains a given set of equations E is just the finest congruence on the term algebra that contains all pairs $\sigma(s) = \sigma(t)$* [in our chosen notation: $\sigma s = \sigma t$]*, for all equations $s = t$ in E and all substitutions σ (the substitution-invariant congruence generated by E).*

Definition 7 (E-Equality (Bürckert et al., 1987))**.** *We say s and t are E-*equal*, abbreviated by $s =_E t$, if and only if the terms s and t are in the congruence. E is a presentation of the congruence "$=_E$" or an axiomatization of the equational theory $T(E)$.*

2.3.5 Anti-unification modulo equational theory

Given first-order terms t_1 and t_2 and an equational theory E, the (first-order) anti-unification problem is that of finding a generalized term t along with substitutions σ_1 and σ_2 that satisfy $\sigma_i t =_E t_i$ for $i \in \{1, 2\}$ where $\sigma_i t$ denotes applying a substitution to a term.

The higher-order problem is equivalent but with higher-order terms mentioned in place of first-order terms.

2.4 Summary

This chapter presented the background for classical terms, substitutions, and formalization of generalization with anti-unification. In the context of anti-unification the concepts of least general generalization and of first-order versus higher-order were discussed also. Then we introduced the extension of anti-unification to equational theories other than syntactic equality, as anti-unification modulo equational theory. Finally, we provided formal definitions of the studied forms of anti-unification.

Chapter 3

Approach

In this chapter we present our approach for comparing different generalization methods of a syntactic representation of source code. We define the mapping of the input representation to an optimized term encoding, and then we use this as the basis for the four forms of anti-unification that we study in this thesis. Then we explain our approach for comparing the forms based on effectiveness and efficiency.

We start in Section 3.1 with an introduction to the Elixir programming language and our approach to representing Elixir source code for the purpose of generalization. We explain our four, studied forms of anti-unification in the context of generalizing Elixir source code in Section 3.2. Then in Section 3.3 we discuss how we compare the different forms based on measures for effectiveness and efficiency. Finally, we summarize in Section 3.4.

3.1 Elixir source code

In order to evaluate the four forms of AU studied in this thesis, we wish to minimize potential complications, biases, and errors in encoding a programming language's syntax as terms. At the same time we would like to reuse performance optimized implementations of operations on these terms. To this end, the Elixir programming language is among the best tools for the job.

The Elixir programming language is a dynamic functional programming language that runs on the Erlang virtual machine. It is popular for implementing scalable soft real-time systems at software companies with massive user bases such as Discord, who claim, "What we do in Discord would not be possible without Elixir."[1]

Although implementing anti-unification for academic purposes may not require particular scalability and

[1] https://elixir-lang.org/blog/2020/10/08/real-time-communication-at-scale-with-elixir-at-discord/

real-time properties, there are other advantages of using Elixir: its underlying highly optimized implementation of terms and pattern matching operations. The language's foundation upon a mathematical theory of terms[2] also leads to simpler source code as will be evident in Chapter 4. The fast performance obtained from the powerful language runtime will be evident from the performance timings in Chapter 5. As well, due to the meta-programming capabilities of Elixir, its built-in source code parsing functions provide a standardized representation of Elixir source code as Elixir terms.

The mathematical basis of terms in Erlang and Elixir is compatible with the classical theory of terms, but the terminology used has some differences related to variables, as clarified in this section.

Erlang defines a fixed set of terms that form the basic data types of programs. Elixir compiles to the same virtual machine code as Erlang and consequently reuses the same underlying implementation of terms from Erlang. Although the set of terms has some configurability at runtime (related to finite memory limits), the set cannot generally be extended by users of the language. In other words, all Erlang and Elixir programs are based upon the same definition of terms,[3] which we will call *Erlang terms*. Examples of these include *atoms* (named symbols), *binaries* (binary numbers), *lists* (linked lists), and representations mapping to standard mathematical concepts — *numbers*, *tuples*, and *maps*.

Before discussing the formalization of representing Elixir source code using a term encoding, we provide a basic overview of Elixir syntax.

3.1.1 Elixir syntax

In Listing 3.1, an example illustrates some Elixir syntax for defining a function. In contrast to statically typed languages with function declarations, Elixir is dynamically typed and provides function *definitions* instead. The key distinction is that a declaration is a statement that declares a new type while a definition in Elixir is an expression that assigns a function name to a clause. A clause is a relation of a *head* pattern (or multiple patterns) and a *body* expression.

We see a function definition (lines 1–6) consisting of two parts, a header (line 1) and a body (lines 1–6).

```
1 def parse(string) do
2   case Code.string_to_quoted!(string) do
3     {:__block__, _, _} = term -> term
4     term -> {:__block__, [], [term]}
5   end
6 end
```

Listing 3.1: Example of Elixir source code involving a function definition and a case expression.

[2] https://www.erlang.org/doc/reference_manual/data_types

[3] This assumes the same version of the Erlang language, as later versions sometimes expand on the term definitions.

The header marks the start of the definition with the keyword **def** followed by the name of the function (parse, in this example) and then its comma-separated list of pattern arguments (only string, in this example) enclosed in parentheses. Function names are allowed to end in exclamation marks or question marks but the meaning of this is only by informal convention (typically an explanation mark signifies functions where failure cases raise an exception and a question mark signifies the function returns a boolean). The body is demarcated by the two keywords **do** and **end**.

Inside the body, we see a **case** expression (lines 2–5) consisting of two parts, a header and a body. The header of the **case** expression (line 2) marks the start of the expression with **case** followed by an expression to evaluate, in this example a call to the function Code.string_to_quoted! passing the argument string that corresponds to a use of the variable defined in the header of the function definition for parse. Inside the body of the **case** expression (lines 2–5), we see two clauses (one on line 3 and one on line 4) of the form "pattern −> result"; the result of the expression to evaluate (string, in this example from the header of the **case** expression) is tested against the pattern in each clause until a match is found, at which point the **case** expression is interpreted as the result from the clause whose pattern matched.

A clause head is a pattern expression followed by "−>" and then the body of the clause. The first pattern here uses a three-tuple with the first element being an atom value :__block__ and the second and third elements being anonymous variables denoted by underscore characters. An atom (denoted with a colon prefix) is a named constant where a constant corresponds with a symbol and the name is an identifier assigned to it. Note that the underscores in :__block__ are part of the name of this atom. The overall **case** expression evaluates to the body of the first matching clause or otherwise raises an exception if none match. Here, the last pattern term serves as a default pattern that will always match: it is a single variable that matches any Erlang term. When matched, the variables become bound and can be referenced in the clause body. A match of the term pattern results in the construction of a three-tuple with term being wrapped into a list, which is denoted by square brackets. This three-tuple is a "quoted expression" (explained in the next section) and its use in this function is to force individual expressions to be treated as blocks; by wrapping an individual expression in a block, we can avoid issues of comparing blocks and non-block expressions.

Just as **case** expressions can have multiple clauses, a function can likewise be defined by multiple clauses

```
1 def parse("")  do
2     :nil
3 end
4 def parse(string) do
5   ...
6 end
```

Listing 3.2: Example of Elixir source code involving a function definition with two clauses.

as shown in Listing 3.2. The first clause (lines 1–3) matches an empty string denoted by two double quotation marks (one opening and one closing); an alternative syntax for the **do** block is used here that allows a single term to be specified as the body (line 2). We elide the body (line 5) of the second clause (lines 4–6) in this example; it could be as shown in Listing 3.1.

3.1.2 Term representation

Because this thesis investigates generalization of source code, we are specifically interested in the syntactic representation of source code as terms. Fortunately, Elixir provides a standardized term representation of Elixir source code as part of its meta-programming capabilities. In particular, the `Code` module includes a function `string_to_quoted!` which converts a string (binary) representation of Elixir source code to an Erlang term representation called a *quoted expression*.[4]

Elixir source code corresponds to expressions, which can contain variables. However, the definition of Erlang terms does not include variable symbols; this is possible in the theory of classical terms by constraining the set of variable symbols to be empty. Terms formed without variable symbols are variable-free terms called *ground* terms (Dershowitz and Jouannaud, 1990). Thus, while an Erlang term is still technically a term, it refers more specifically to a ground term. When representing variables in the source code expressions, these map to function terms instead of single variable symbols.

To encode the syntactic representation of a variable as an Erlang term, Elixir treats a variable as a *variable expression* encoded as a tuple containing an identifier name and a *context* name. The identifier name corresponds with a textual representation of the variable expression in source code. Contexts are named namespaces that expose and group functionality; use of the context name allows disambiguation of different variable expressions possessing identical identifier names.

When Elixir source code is compiled, the context of every variable expression is embedded into the term encoding of the variable expression; the notion of a variable without a context is meaningless. If we take away the context name then we are left with just an identifier name, which would be encoded as an atom. In the opposite direction, starting with just a name we can add to it a context name in order to form a variable. A context can be thought of as tracking the binding of names to Erlang terms (the ground terms that a variable expression evaluates to) and this variability in binding provided by contexts is what makes the variable expressions actually variable. For example, the code "`Code.eval_quoted(quote(do: var!(x)), x: 1)`" evaluates the quoted expression `var!(x)` with the binding `x: 1` that acts as a context mapping the name `x` to the ground term `1`. The `var!` macro is necessary to mark the variable as not being "hygienized"; otherwise the two occurrences of "`x`" would correspond with different contexts (thus signifying different variables).

[4] https://hexdocs.pm/elixir/Code.html#string_to_quoted!/2

3.1.3 Compact quoted expression

Elixir's quoted expressions are not ideal for being generalized directly using anti-unification algorithms because (1) they embed metadata pertaining to syntactic representation and (2) they use data structures not optimized for performance. Instead, we use an altered version instead that we refer to as a *compact quoted expression*.

To encode the syntactic representation of a function call, Elixir treats a function call as a *function call expression* denoted by a name and a list of arguments. Note that a function symbol from the set of function symbols in the Erlang term model cannot be used to encode the expression because Erlang terms do not permit such user-defined function symbols. Instead, a tuple term is used to capture the function name (or name expression) along with a list of its arguments recursively encoded as quoted expressions.

In the compact quoted expression's encoding, we wish to include the arity of a function so that it does not need to be re-computed each time from the list of arguments. Due to the logical definition of lists (i.e., as a linked list), the cost of computing the length is proportional to the length. To avoid this repeated cost, we simply embed the arity as an integer into the representation.

To represent function call expressions as quoted expressions, Elixir uses a tuple with three elements consisting of (1) an atom or other tuple, (2) metadata, and (3) arguments for the function call. As mentioned earlier, variables are not Erlang terms but instead expressions and they are represented similarly to function calls with two differences: (1) the first element is an atom encoding the name of the variable, and (2) the arguments are replaced by an atom representing the variable's context. The context is critical because variable names within an expression can originate from different modules of source code (as a result of meta-programming). Without a context there is a potential problem of naming collisions between different modules. Hence, the context is necessary to disambiguate the name.

Listing 3.3 compares the quoted expression form with our compact form for the Elixir expression $f(:a, b)$. The definition of string_to_compact_quoted is ignored here but will be provided in Chapter 4.

In line 1, the input expression as a string value is bound to the variable string_expr by the use of the pattern matching operator "=".

In line 2, the string is used to obtain the quoted expression using the built-in Code module. The "=" operator is used stylistically here to convey that the right-hand side is expected to match the literal on the

```
1 string_expr = "f(:a, b)"
2 {:f, [line: 1], [:a, {:b, [line: 1], :nil}]} = Code.string_to_quoted!(string_expr)
3 {:f, 2, [:a, {:b, 0, []}]} = string_to_compact_quoted(string_expr)
```

Listing 3.3: Comparison of a quoted expression and a compact quoted expression.

left-hand side. In the quoted form (line 2), we see that the metadata [line: 1] gives syntactic information related to the whitespace formatting within the source code string. We also see that the variable expression b in the source code string maps to the quoted form {:b, [line: 1], :nil} with the name b being encoded as the atom :b. Here the context is shown as :nil, but this would be replaced by the name of the context if it were included in an environment argument passed to the string_to_quoted! function.

We contrast this quoted form with the term in line 3 that corresponds with our compact quoted expression form. The main difference here is that the metadata [line: 1] in the quoted expression is replaced in the compact form with the relevant arity. As well, the context of :nil is converted to an empty list. This treats variables as function calls of arity zero. Although this is not an ideal approach to encoding since variables and function calls are different, it works in practice because real-world source code should not have naming collisions between variable expressions and the names of functions of arity zero.

Quoted expressions use three-tuples to encode function calls and variable expressions but this leaves the problem of how to encode "literal" three-tuples in the quoted form. For this the function named {} is used to provide an "escape." For example the Elixir expression {1,2,3} becomes encoded as the quoted expression {:{}, [], [1, 2, 3]}. This is actually the same for all other tuples except for two-tuples. Two-tuples in the Elixir expression are still two-tuples in the quoted form. Most literal terms including lists, atoms, binaries, and numbers have no special treatment in the quoted form. In addition to tuples (other than two-tuples) there are also maps that become escaped. For example, the map %{a: 1} is quoted as {:%{}, [], [a: 1]}.

Realizing compact quoted expressions. Let $\mathbb{T}_c(\mathbb{V}_c, \mathbb{F}_c, \mathbb{FV}_c)$ be the set of terms represented by compact quoted expressions, where $\mathbb{V}_c$ is the set of term variable symbols, $\mathbb{F}_c$ is the set of function symbols (each with a specific arity), and $\mathbb{FV}_c$ is the set of function variable symbols (each with a specific arity).

We realize the set of function symbols $\mathbb{F}_c$ by deriving them from the quoted expression terms that represent function calls and those that are literals. This set of function symbols is needed later to define terms that use these symbols. Let T_q be a set of quoted expressions, which are first-order ground terms. The first element in the three-tuple forms (representing function call expressions) is included in a set of function symbols $F_n = \{f : \langle f, meta, [t_1, ..., t_n]\rangle \in T_q\}$ where angle brackets denote tuples, *meta* is a keyword list used by different parts of the Elixir compiler,[5] $[t_1, \ldots, t_n]$ is a list of terms from T_q representing the arguments to the function call, and n is the arity. For example, the expression M.f(1) is represented as a quoted expression three-tuple ⟨.(aliases(M), :f), [ ], [1]⟩ and so we take the first element .(aliases(M), :f) as a function symbol (of arity one). Next, we handle literals in the quoted expression terms (i.e., functions of arity zero). Since these can overlap with the function symbols derived from function call expressions, there is a choice of whether to

[5] https://hexdocs.pm/elixir/Macro.html#t:metadata/0

allow the overlaps or to keep them distinct. For the sake of managing scope, we have chosen to keep them distinct and leave the exploration of alternative choices as future research. For this reason, we introduce a set of literal or constant symbols that are distinguished from F_0: let L be the set of literals such that $\forall l \in L, l \notin \bigcup_{n \geq 0} F_n \wedge l \notin \mathbb{V}_c \wedge l \notin \mathbb{FV}_c$. Then we join all these together to form the complete set of function symbols: $\mathbb{F}_c = L \cup \bigcup_{n \geq 0} F_n$.

We define the set of variables, $\mathbb{V}_c$, and sets $\mathbb{FV}_c^m$ of m-ary function variables for the compact quoted expression terms using two-tuple representations. These variables do not occur in the original compact quoted expression terms of the input dataset and may only be formed after applying anti-unification. Since every variable originates from generalizing two (ground) compact quoted expression terms, we take advantage of this in defining the sets of variables by $\mathbb{V}_c = \{\langle t_1, t_2 \rangle \mid t_1, t_2 \in \mathbb{T}_c(\emptyset, \mathbb{F}_c, \emptyset)\}$ and $\mathbb{FV}_c^n = \{\langle f_1, f_2 \rangle \mid f_1, f_2 \in F_n\}$, where $\mathbb{FV}_c = \bigcup_{n \geq 0} \mathbb{FV}_c^n$. For example, anti-unifying $\mathsf{f(1)}$ and $\mathsf{g(2)}$ can result in an anti-unifier $\langle f, g \rangle(\langle 1, 2 \rangle)$ in the higher-order case and $\langle f(1), g(2) \rangle$ in the first-order case.

By putting these together as per the higher-order term definition, the set of compact quoted expressions $\mathbb{T}_c(\mathbb{V}, \mathbb{F}, \mathbb{FV})$ is defined as the least set satisfying: (1) $\mathbb{V}_c \subseteq \mathbb{T}_c$; (2) for all n, for all terms $t_1, ..., t_n$ and for all n-ary function symbols $f \in \mathbb{F}_c^n$: $f(t_1, ..., t_n) \in \mathbb{T}_c$, where $\mathbb{F}_c = \bigcup_{n \geq 0} \mathbb{F}_c^n$; and (3) for all m, for all terms $t_1, ..., t_m$, and for all m-ary function variable symbols $F \in \mathbb{FV}_c^m$: $F(t_1, ..., t_m) \in \mathbb{T}_c$, where $\mathbb{FV}_c = \bigcup_{n \geq 0} FV_c^n$.

An important limitation in this approach is that we only handle generalizing two ground terms and leave the case of further "iterative" generalization as out of scope. As well, the binary (i.e., string) representation of generalized compact quoted expressions (i.e., with variables) using Elixir syntax is out of scope. A discussion of the complications related to these issues is left for Chapter 6.

This notation for the sets of terms will be used in the next section to explain the implementations for the different forms of anti-unification.

3.2 Anti-unification forms

The four forms of anti-unification considered in this thesis are first-order (FO), higher-order (HO), first-order modulo equational theory (FO+ET), and higher-order modulo equational theory (HO+ET). The implementations of these will be discussed in Chapter 4 while the general approach is discussed here.

3.2.1 First-order and higher-order forms

The first-order form of anti-unification applied to two ground terms is modelled by the function $\mathbb{T}_c(\emptyset, \mathbb{F}_c, \emptyset) \times \mathbb{T}_c(\emptyset, \mathbb{F}_c, \emptyset) \rightarrow \mathbb{T}_c(\mathbb{V}_c, \mathbb{F}_c, \emptyset)$ that finds a first-order LGG given two ground terms.

For the higher-order form, the function we are interested in implementing is $\mathbb{T}_c(\emptyset, \mathbb{F}_c, \emptyset) \times \mathbb{T}_c(\emptyset, \mathbb{F}_c, \emptyset) \rightarrow$

$\mathbb{T}_c(\mathbb{V}_c, \mathbb{F}_c, \mathbb{FV}_c)$. This models the application of higher-order anti-unification on two ground terms, producing an LGG that can include higher-order terms.

Note that, for the case where the arity of functions differ, we require an equational theory to find a generalization between the argument lists of differing length. So for higher-order anti-unification without equational theory, the LGG for function terms of different arity such as $f(a)$ and $g(a, b)$ would still be just X. In this thesis, we do not define an equational theory to allow this for arbitrary function symbols; however, for the __block__ symbol we do allow generalization when their arities differ: the equational theory for this symbol is given next.

3.2.2 Equational theory

Table 3.1 shows the formalized equational theory used in this thesis. A __block__ is a function symbol originating from the quoted expression form of blocks of expressions. These occur in module and function definitions as well as other expressions such as **if** expressions. There are four equations in this formal definition which cover all possible cases of how the **nil** term can be embedded.

The first equation shows that an empty block is equal to a block containing only the **nil** term. In the second equation a non-empty block is equal to one with the **nil** term prepended. In the third equation a non-empty block is equal to one with the **nil** term appended. Finally, the fourth equation shows a block with at least two elements is equal to one with the **nil** term inserted somewhere between the elements.

Arity and the `__block__` Function Symbol. The symbol `__block__` is used pervasively in quoted Elixir expressions to represent sequences of code statements grouped as a block. While a single function symbol is used in the source representation, our formal model interprets this symbol as a *family of function symbols*, each corresponding to a specific arity. That is, each term of the form `__block__`$(t_1, \ldots, t_n)$ is assigned an implicit function symbol `__block__`^{n}, where n is the number of arguments.

This disambiguation is necessary to support reasoning about equality and substitution between block expressions of different lengths. For instance: `__block__`() is interpreted as the nullary function symbol `__block__`^{0}, `__block__`(t) as `__block__`^{1}, and so on for increasing arity.

Table 3.1: Equational theory for __block__ and **nil**.

$\textsf{__block__}() =_{block} \textsf{__block__}(\textbf{nil})$
$\textsf{__block__}(t_1, t_2, ..., t_n) =_{block} \textsf{__block__}(\textbf{nil}, t_1, t_2, ..., t_n)$, for all $n \geq 1$
$\textsf{__block__}(t_1, t_2, ..., t_n) =_{block} \textsf{__block__}(t_1, t_2, ..., t_n, \textbf{nil})$, for all $n \geq 1$
$\textsf{__block__}(t_m, t_{m-1}, ..., t_1, u_1, u_2, ...u_n) =_{block} \textsf{__block__}(t_m, t_{m-1}, ..., t_1, \textbf{nil}, u_1, u_2, ...u_n)$, for all $m, n \geq 1$

This arity-specific interpretation allows us to formulate an *equational theory* in which blocks of differing arity can be related through neutral insertions of the `nil` symbol. These insertions serve as identity elements in the alignment of sequence structures, preserving semantic equivalence while enabling generalization.

The equational theory for `__block__` expressions is shown in Table 3.2, and is used to equate blocks that differ by inserted `nil` terms. These rules reflect the intuition that `nil` plays the role of a no-op element in code, and that many structural similarities between blocks are obfuscated by irrelevant or absent statements. Our theory accounts for this via the Needleman–Wunsch alignment process described in Section 4.2.2.

Table 3.2: Equational theory for `__block__` and `nil`.

$$\texttt{__block__}() =_{\text{block}} \texttt{__block__}(\text{nil})$$
$$\texttt{__block__}(t_1, \ldots, t_n) =_{\text{block}} \texttt{__block__}(\text{nil}, t_1, \ldots, t_n), \quad \text{for all } n \geq 1$$
$$\texttt{__block__}(t_1, \ldots, t_n) =_{\text{block}} \texttt{__block__}(t_1, \ldots, t_n, \text{nil}), \quad \text{for all } n \geq 1$$
$$\texttt{__block__}(t_1, \ldots, t_m, u_1, \ldots, u_n) =_{\text{block}} \texttt{__block__}(t_1, \ldots, t_m, \text{nil}, u_1, \ldots, u_n), \text{ for all } m, n \geq 1$$

As a final note, the $\texttt{__block__}^n$ notation is not used explicitly in our term representations, but it is conceptually useful for grounding the formal model. Internally, our implementation treats each `__block__` expression as an application of a function symbol whose arity is inferred directly from the structure of the term.

Our approach to implement a search for an LGG that takes advantage of this equational theory is based on the popular Needleman–Wunsch method (Needleman and Wunsch, 1970). It gives us a dynamic programming formulation for computing optimal alignment scores that has been widely adopted for aligning samples of genomic sequences. It involves scoring the different possible alignments of two sequences where gaps can be introduced between elements. In our approach, gaps in a sequence of terms correspond with **nil** terms added to a block in accordance with our equational theory. The alignment with the highest score is the one that we are particularly interested in. Calculating the score is based on counting the number of syntactic commonalities in the candidate generalized term. A measure called *non-variable count* corresponds with this count and is explained in the next section. Details of the score along with its integration into the Needleman–Wunsch implementation are given in Chapter 4.

This approach to sequence alignment allows finding a generalization of sequences that maximizes the number of commonalities. However, this is not necessarily the best *quality* generalization, since quality can be defined in ways other than just maximizing the count of symbolic commonalities. This problem is well known in genomic analysis and has many aspects including the problem that global sequence alignment will "frequently align unrelated regions" (Miller, 2001). Overcoming this problem involves more sophisticated statistical modelling to better define "quality." An example of this in source code analysis is probabilistic modelling for API sequence pattern mining (Fowkes and Sutton, 2016). Local sequence alignment methods

also fall into this category of more sophisticated approaches. In terms of maximizing the number of non-variable symbols in the generalization, however, global sequence alignment finds an optimal solution under the constraint that every position in both sequences must be aligned.

In addition to this constraint, the Needleman-Wunsch method operates under several assumptions:

1. **Additivity**: The total alignment score is the sum of individual position scores:

$$S(A) = \sum_{i=1}^{n} s(a_i, b_i) \tag{3.1}$$

   where $S(A)$ is the total alignment score, n is the alignment length, and $s(a_i, b_i)$ is the score for aligning terms a_i and b_i at position i.

2. **Independence**: The score at each position is independent of scores at other positions:

$$s(a_i, b_i) \perp s(a_j, b_j) \quad \forall i \neq j \tag{3.2}$$

3. **Global optimality constraint**: Every term in both sequences must be aligned.

The Needleman-Wunsch method provides the following guarantees:

1. **Global optimality**: The solution maximizes the scoring function:

$$A^* = \arg\max_{A \in \mathcal{A}} S(A) \tag{3.3}$$

   where A^* is the optimal alignment and $\mathcal{A}$ is the set of all possible global alignments.

2. **Completeness**: Every term in both input sequences appears in the final alignment:

$$|A| = |X| + |Y| - \text{matches} \tag{3.4}$$

   where $|A|$ is alignment length, $|X|$ and $|Y|$ are sequence lengths.

3. **Existence**: An optimal alignment always exists:

$$\exists A^* \in \mathcal{A} : S(A^*) \geq S(A) \quad \forall A \in \mathcal{A} \tag{3.5}$$

For this thesis, we avoid the complexities of different definitions of quality that local sequence alignment can address and instead focus on relative comparisons of effectiveness and efficiency between a selection of

different forms of anti-unification as explained in the next section. A comparison of different quality measures is, however, a promising direction for future research.

3.2.3 Validation of implementation

To validate that the implementation of the Needleman-Wunsch method computes the optimal alignments properly we performed an exhaustive comparison on some (non-trivial) example terms. Due to the simplicity of defining the forms of anti-unification without equational theory in Elixir, we assume after carefully checking these (seven lines of source code) that they are correct. Then we enumerated all candidate alignments by inserting the possible combinations of gaps into the sequences of the example terms. The generalizations for all of these aligned terms were then computed and the optimal result found through this exhaustive search was compared with the result from the Needleman-Wunsch method. Because the results matched for the examples considered we have a high confidence in the correctness of the implementation.

3.2.4 Combinations

Our equational theory we defined in Table 3.1 for blocks of statements can be used with first-order anti-unification as there is no dependence on higher-order terms. We therefore have four total possible combinations which, along with the abbreviation that will be used, are as follows: (1) first-order without equational theory (FO), (2) higher-order without equational theory (HO), (3) first-order with equational theory (FO+ET), and (4) higher-order with equational theory (HO+ET).

We study these four forms in this thesis; we explain our approach to comparing them in the next section.

3.3 Comparing anti-unification forms

In this section we discuss our approach to comparing the four forms of anti-unification that we have introduced.

Different forms of anti-unification can result in different LGGs capturing varying quantities and kinds of commonalities. We consider the goal of applying anti-unification to centre on finding many relevant commonalities between the input terms. The count of these commonalities found in the computed LGG using a particular form of anti-unification is considered the *effectiveness*. Computing the LGG also comes with a *cost* that we will measure in terms of the execution time taken (i.e., performance). When effectiveness is normalized by this cost we obtain *efficiency* as per the equation "efficiency = effectiveness/cost."

Another way to conceptualize the efficiency is as a rate of symbolic commonalities detected in units of symbols per second. Note that symbols are treated as the most basic unit, and each pair of identical symbols

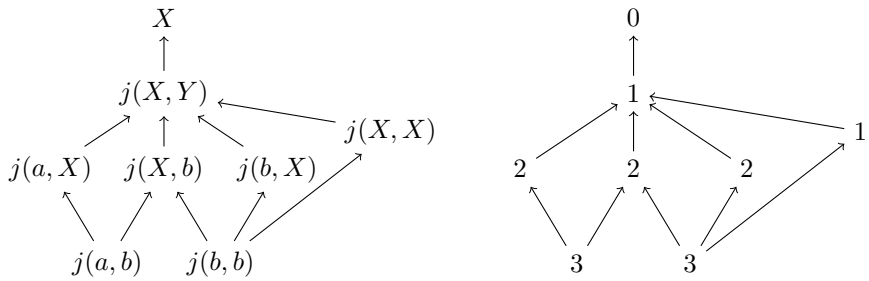

Figure 3.1: Generalized terms of two input terms $j(a,b)$ and $j(b,b)$ with their corresponding non-variable counts on the right.

contributes equally to the efficiency regardless of what the symbols represent. This has some limitations which we discuss in Chapter 6.

This approach allows a measure of effectiveness and efficiency to be obtained for each anti-unifier (generalized term) being computed. This set of anti-unifiers comes from the dataset that is chosen for experimental study. Our dataset used in this thesis is discussed in Chapter 4. Using the dataset, the experimental results produce the two measures of effectiveness and efficiency for each of the four different forms repeated for a subset of pairs of input terms in the dataset. In Chapter 5 these results will be presented and analyzed to provide both an overall comparison between the forms as well as a comparison between the distributions of measurements.

Next we discuss further details for the measurements including how commonalities are counted to obtain effectiveness and how the performance is timed to obtain efficiency.

3.3.1 Commonality counting

Terms are composed of symbols that are categorized as variable symbols, function symbols, and constant symbols. Since our inputs are ground terms, a generalization formed from these will have variables that correspond with absence of commonality while the rest (constants and non-variable function symbols) correspond with a captured commonality. To refer to the symbols that are not variables, we will use the term *non-variable* (symbols) and refer to the quantity of these in a term as *non-variable count* (NVC).

In Figure 3.1 the three generalizations $j(X,b)$ (which is the LGG), $j(X,Y)$, and X are shown to have non-variable counts of 2, 1, and 0 respectively. While this works well for this case, there is a limitation for $j(X,X)$ and $j(X,Y)$ which both have the same non-variable count of 1. However, $j(X,X)$ is more specific and captures an additional commonality that $j(X,Y)$ does not. Because of this limitation the precision of the NVC measure is not ideal. Unfortunately, the complexity of considering both of these distinct kinds of commonalities necessitated leaving it as future work. A brief explanation of this challenge follows to give a

sense of the additional complexity that would arise.

The presence of shared variables in a term $j(X, X)$ corresponds with a notion of the term's self-similarity or "intra-term" similarity — commonalities *within* the generalized term — while the NVC corresponds with "inter-term" similarity — commonalities *between* the input terms. Combining these two independent quantities requires a multi-dimensional value composed of two scalar values. While some of the multi-dimensional values would be comparable (e.g., $\langle 1, 2\rangle < \langle 0, 2\rangle$), others would not be (e.g., $\langle 1, 2\rangle$ and $\langle 2, 1\rangle$).

3.3.2 Performance timing

Computing the LGG requires computational resources. These multiple costs associated with running anti-unification algorithms include memory space, memory access, and central processing unit (CPU) cycles. For simplicity, this thesis considers only the CPU cost in terms of time. However, memory access inevitably must take place and can introduce significant variance. Specific techniques were used to minimize the memory access cost as much as possible and these are discussed in Chapter 4.

An important aspect of measuring the cost of anti-unification is starting the timing from when the anti-unifier actually starts being computed and excluding any overhead before that point such as loading of the executable program or starting a language's runtime dependencies. As well, the cost of transforming data into the necessary representation used by the anti-unification algorithm is not counted towards the performance cost. For this thesis this means we treat the cost of encoding a binary representation of source code into the compact quoted expression form as an amortized operation and therefore omitted from the performance timing.

To measure the performance we use Erlang's internal function :erlang.monotonic_time(:nanosecond) to measure the elapsed time of our anti-unification implementing function.[6] Due to randomness in the measured timings (from variance in CPU scheduling, for example) we sample multiple runs and use the median of the timings in order to reduce the variance. Sampling multiple runs is done by repeating the round of measurements across the dataset and using a randomized order each time. The method of determining a sufficient number of samples is provided in Chapter 4 along with the result.

3.4 Summary

In this chapter, we presented our approach for generalizing a syntactic representation of Elixir source code. We introduced the formulations for the different forms of anti-unification based on the compact quoted expression representation. This included first-order versus higher-order as well as syntactic versus modulo

[6] Specifically the :timer.tc function is used which internally uses monotonic time (`https://www.erlang.org/doc/man/timer`).

equational theory (using a specific equational theory for blocks). We explained our approach to comparing these forms using measures for effectiveness and efficiency, along with providing explanations of the performance time and non-variable count measures.

Chapter 4

Methodology

In this chapter, we provide a detailed description of the experimental process used in comparing the effectiveness and efficiency for the four different forms of anti-unification being studied.

In Section 4.1 the implementation of encoding source code as terms is discussed. Section 4.2 describes the implementation of the four anti-unification forms. Sections 4.3 and 4.4 cover the implementation of the non-variable count and performance measures. Section 4.5 describes the input data that we use. Lastly, Section 4.6 summarizes.

4.1 Encoding source code as terms

Since anti-unification methods operate on terms, the source code needs to be parsed and converted to a term. As discussed in Chapter 3, we use the Elixir parser but transform the resulting quoted expression into our compact quoted expression form before applying anti-unification.

Listing 4.1 shows the parsing and term transformation implementations. The function string_to_compact_quoted accepts a string as an input and returns the compact quoted representation. It applies two other functions to the string, parse and to_compact, which are defined on lines 5 and 12 respectively.

Going line by line, we start the definition for string_to_compact_quoted on line 1. This function is responsible for taking a source code string as input and transforming it into a compact term representation. On line 2, the parse function is called to first convert the string input into a quoted Elixir term representation. After that, the output of parse is passed to the to_compact function to further "compact" the term. On line 3, **end** marks the end of the string_to_compact_quoted function definition.

On line 5, we start the definition for the parse function, which is responsible for converting a string of Elixir code into a quoted Elixir term representation. The function takes one argument, string, which is a

```
def string_to_compact_quoted(string) do
  to_compact(parse(string))
end

def parse(string) do
 case Code.string_to_quoted!(string) do
  {:__block__, _, _} = term -> term
  term -> {:__block__, [ ], [term]}
 end
end

def to_compact(elixir_term) do
 case elixir_term do
  {a, b} ->
   to_compact(quote do
    unquote(:{})(unquote(a), unquote(b))
   end)

  list when is_list(list) ->
   to_compact(quote do
    unquote(:"[]")(unquote_splicing(list))
   end)

  {name, _, namespace} when is_atom(name) and is_atom(namespace) ->
   to_compact(quote do
    unquote(name)()
   end)

  {function_symbol, _, args} when is_list(args) ->
   {function_symbol, length(args), Enum.map(args, &to_compact/1)}

  t -> t
 end
end
```

Listing 4.1: Elixir implementation of parsing source code into a representation of an anti-unification term.

string of Elixir source code. Line 6 calls the Code.string_to_quoted! function, which parses the Elixir code provided in the string and returns its quoted abstract syntax tree (AST) representation; the ! signifies that it will raise an exception if there is an error in parsing the string. The result of this parsing is then matched against patterns in the case expression. On line 7 is the first pattern in the case expression; it matches the quoted representation of a block of code, which is represented by the atom :__block__ in Elixir ASTs. If the parsed code is already a block (represented as {:__block__, meta, body}), then it simply returns this term unchanged, without using the meta and body parts. On line 8 is the second pattern which matches any other term (term); if the parsed string is not already a block, this line ensures that it is wrapped into a block by creating a tuple: {:__block__, [], [term]} —- the empty list "[]" represents an empty metadata field, and [term] is a list containing the parsed term. This ensures that the parsed term is always treated as a block even if it originally was not one. On line 9, **end** marks the end of the case expression and on line 10 **end** marks the end of the definition of parse.

On line 12, we start the definition for the to_compact function which transforms the quoted Elixir term into a more compact representation. The function takes one argument, elixir_term, which represents the quoted Elixir term that needs to be transformed into a compact form. Line 13 starts a case expression that matches the shape and structure of the elixir_term and performs specific transformations based on this, as follows. On line 14 the {a, b} pattern matches a two-element tuple where a and b represent the two elements of the tuple. On line 15 we recursively call to_compact after transforming the two-element tuple into a quoted expression (with quote **do** ... **end**) of a function application :{} to the two arguments a and b (line 16). The quote **do** ... **end** macro (lines 15–17) forms a quoted expression — similar to the string_to_quoted! function — from its argument expression. Elixir values are interpolated into the quoted expression using the unquote and unquote_splicing function expression. This is used to form function expressions with {} and [] as the function names. For example, a function call to a function named {} is represented by the quoted expression {:{}, [], []} but the syntax of the Elixir language does not (yet) support this kind of expression (i.e., "{}()" is not syntactically valid). So in order to form such an expression we need to interpolate the value ":{}" (the name of the function encoded as an atom value) into the quoted expression (the three-tuple form). The "unquote" macro (macros are functions that typically run during compilation) allows us to perform this kind of interpolation or escaping of values within a quoted expression. On line 17 **end** marks the end of the quote **do** block.

On line 19 the pattern list **when** is_list(list) matches if elixir_term is a list because the guard **when** is_list(list) ensures that only lists are matched by this clause. Similarly to the tuple case, line 20 recursively calls to_compact after transforming the list into a quoted expression using quote **do** ... **end**. Inside the quoted expression, on line 21 the list is transformed into a function application of [] to the arguments extracted from

the list using unquote_splicing.

On line 24, the pattern {name, _, namespace} with the guard **when** is_atom(name) **and** is_atom(namespace) matches three-element tuples where name and namespace are atoms. As the middle element is metadata, it is ignored here via "_". This quoted expression corresponds with a variable expression; it is transformed on line 26 into a function application of a function with a name equal to the value bound to the variable name. On line 27, **end** marks the end of the quote **do** block.

Function application quoted expressions are handled on line 29. Here the pattern {function_symbol, _, args} matches three-element tuples representing function applications where _ is the metadata (ignored) and args is a list of function arguments (the guard is_list ensures only lists match). On line 30 a three-tuple representing a compact quoted expression is returned where function_symbol is the term representing the function symbol, length(args) computes the arity of the function from the length of arguments, and Enum .map(args, &to_compact/1) recursively applies the unary to_compact function to each argument in the args list.

Any quoted expression that did not match any previous patterns will be handled by the pattern on line 32. Here the pattern t matches any term and returns it unchanged; these should be literals such as atoms, numbers, and binaries.

The **end** on lines 33 and 34 mark the end of the case expression and to_compact function definition respectively.

After the transformation implemented by to_compact, the resulting compact expression will consist of only function terms encoded as three-tuples and literals (atoms, numbers, and binaries).

4.2 Anti-unification implementation

The implementation for each form of anti-unification is provided in the listings of this section. As explained in Chapter 3, to represent variable symbols in the generalized term we use a two-tuple encoding since this set will still be disjoint from the set of function symbols and the set of ground terms.

4.2.1 First-order and higher-order

We implement the first-order form according to the algorithm for syntactical generalization given by Burghardt and Heinz (2014). Using Elixir we require three clauses as shown in Listing 4.2.

For the first clause (line 1), the function symbol bound to the variable f must be the same in both parameters and in both arities, while the arguments bound to args1 and args2 are allowed to differ. The result for this case is constructed by use of the Enum.zip_with function which applies the anti-unification

```
1 def fo({f, n, args1}, {f, n, args2}), do: {f, n, Enum.zip_with(args1, args2, &fo/2)}
2 def fo(x, x), do: x
3 def fo(x, y), do: {x, y}
```

Listing 4.2: Elixir implementation of first-order anti-unification.

```
1 def ho({f, n, args1}, {f, n, args2}), do: {f, n, Enum.zip_with(args1, args2, &ho/2)}
2 def ho({f, n, args1}, {g, n, args2}), do: {{f,g}, n, Enum.zip_with(args1, args2, &ho/2)}
3 def ho(x, x), do: x
4 def ho(x, y), do: {x, y}
```

Listing 4.3: Elixir implementation of higher-order anti-unification.

binary function fo pairwise to the elements. A reference to the function is denoted by &fo/2 where the number following the slash encodes the arity of the function (note that the ampersand is unrelated to pointers and the reference does not behave like a pointer). For example, zip_with([1, 2], [:a, :b], &fo/2)) is equivalent to [fo(1, :a]), fo(2, :b)].

The second clause (line 2 of Listing 4.2) will be tried for a match if the first one cannot be matched. This clause requires both parameters to be strictly equal syntactically, and then the result is the value bound to the parameter x.

If both of the two prior clauses do not match then the final clause (line 3) will match and return a two-tuple (our encoding for variable symbols) formed with the values bound to the function parameters x and y.

For the higher-order form shown in Listing 4.3, there is an additional clause (line 2) added that handles the case of differing function symbols having the same arity, resulting in the use of a function variable which we represent with the tuple {f,g}. Note that clauses 1 and 2 both make use of the binary function ho.

4.2.2 Needleman–Wunsch alignment

Adding the equational theory for blocks means we can find better generalizations that align expressions. For example, the first-order and higher-order generalizations for __block__(1, 2) and __block__(2, 1) result in an LGG of __block__(X, Y). However, with our equational theory, __block__(1, 2) is equal to __block__(nil, 1, 2) and __block__(2, 1) is equal to __block__(2, 1, **nil**). This makes __block__(X, 1, Y) a valid generalization and it preserves more information (1 is common between the terms). Alternatively, we can add **nil** padding differently and obtain a generalization of __block__(X, 2, Y). Finding the positions to add gaps (i.e., **nil** terms) is equivalent to the problem of sequence alignment and so we use an adapted Needleman–Wunsch algorithm to compute an optimal sequence alignment (note that as the previous example illustrates there

may be more than one optimal solution).

To implement the equational theory, we first implement a function that finds an alignment based on the Needleman–Wunsch algorithm as discussed previously. Listing 4.4 shows this. In order for this function to be general enough for both FO+ET and HO+ET forms we use the antiunify parameter to pass the function that implements anti-unifying terms to compute an LGG. This allows reusing the Needleman–Wunsch implementation for the first-order and higher-order forms.

The first part defines multiple function clauses to handle different base cases. On line 1, we return an empty list if both sequences are empty. This handles the base case where no alignment is needed.

On lines 2 and 3, we handle cases where only one of the sequences is empty. If the first sequence is empty, we pad it with a **nil** and proceed with the recursive call. Similarly, if the second sequence is empty, we also pad it with **nil**. This ensures that gaps will be inserted in the alignment even when one sequence is empty.

On line 4, we have the main function that computes the alignment between two non-empty sequences, args1 and args2, which are our input terms. It follows these steps: (1) tuple conversion and matrix initialization, (2) dynamic programming table creation with scoring and direction calculation, and (3) traceback.

On lines 5 and 6, we convert the input lists args1 and args2 into tuples, t_args1 and t_args2, respectively, for efficient element access, via a standard function provided for List. On lines 8 and 9, we obtain the last index (length of sequence minus one, due to zero-based indexing) for each tuple and assign these to variables n1 and n2.

Starting on line 11, a for-loop iterates over each cell of a 2D matrix (grid) of size $n1 + 1$ by $n2 + 1$. The result will be a grid (dictionary) where the keys are coordinates (x, y) and the values are the optimal scores and directions for aligning elements at positions x and y in args1 and args2. The for-loop iterates over the Cartesian product of the two ranges in x <− 0..n1, y <− 0..n2 and y. The reduce: %{} parameter specifies that an accumulator initialized to an empty map will be passed to the body of the for-loop; on line 12 we bind that accumulator to the variable acc. The computed score and direction is bound to result on line 13 and then put into the accumulator map on line 26. This new accumulator (the result of evaluating the body) is then the accumulator in the next iteration of the loop and the loop repeats until all of the xy-coordinates have been processed. The final result is bound to the variable grid on line 11.

For each cell (x, y) in the grid, we compute the best score by considering three possible moves to other cells in the grid: $\{1, 1\}$, a diagonal move; $\{0, 1\}$, an up move; and $\{1, 0\}$, a left move. The diagonal move represents aligning the term from both sequences. The score for this move is based on the non-variable count in the LGG computed from the terms at these positions. For the up and left moves, these correspond to introducing gaps (i.e., **nil**) and so the score should be zero. Since the grid starts from the top-left corner (0,0) and grows towards the bottom-right, moving left in this grid means decrementing the x-index, just

```
def nw([], [], _), do: []
def nw([], args2, antiunify), do: nw([nil], args2, antiunify)
def nw(args1, [], antiunify), do: nw(args1, [nil], antiunify)
def nw(args1, args2, antiunify) do
  t_args1 = List.to_tuple(args1)
  t_args2 = List.to_tuple(args2)

  n1 = tuple_size(t_args1)−1
  n2 = tuple_size(t_args2)−1

  grid = for x <− 0..n1, y <− 0..n2, reduce: %{} do
    acc −>
      result =
        [{1, 1}, {0, 1}, {1, 0}]
        |> Enum.map(fn
          {1, 1} −>
            au = antiunify.(elem(t_args1, x), elem(t_args2, y))
            score = acc |> Map.get({x−1, y−1}, {0}) |> elem(0)
            {score + nvc(au), {1, 1}}

          {dx, dy} −>
            score = acc |> Map.get({x−dx, y−dy}, {0}) |> elem(0)
            {score, {dx, dy}}
        end)
        |> Enum.max_by(&elem(&1, 0))
      Map.put(acc, {x, y}, result)
  end

  {n1, n2}
  |> Enum.unfold(fn
    {−1, −1} −> nil
    {−1, y} −> {{nil, elem(t_args2, y)}, {−1, y−1}}
    {x, −1} −> {{elem(t_args1, x), nil}, {x−1, −1}}
    {x, y} −>
      delta = grid |> Map.fetch!({x, y}) |> elem(1)
      case delta do
        {1, 1} −>
          {antiunify.(elem(t_args1, x), elem(t_args2, y)), {x−1, y−1}}
        {1, 0} −>
          {{elem(t_args1, x), nil}, {x−1, y}}
        {0, 1} −>
          {{nil, elem(t_args2, y)}, {x, y−1}}
      end
  end)
end
```

Listing 4.4: Elixir implementation of sequence alignment based on Needleman–Wunsch.

as moving up means decrementing the y-index. So, our "left move" actually refers to a move toward the previous cell horizontally.

On line 14 we use the list [{1, 1}, {0, 1}, {1, 0}] to represent the three possible moves (diagonal, up, and left) and pipe (|>) these to a map function on line 15 which will compute a score for each move. The |> symbol takes the value on the left (the move directions list) and passes it as the first argument to the function on the right Enum.map. This is a common way to chain functions in Elixir.

On line 15 the **fn** ... **end** block in Enum.map defines an anonymous function that operates on each move {dx, dy}. For a diagonal move ({1, 1}), this corresponds to aligning the current elements of both sequences. We compute the anti-unifier for elements at (x, y) on line 17. The previous score is retrieved on line 18 and then summed with the score from the anti-unifier computed with the non-variable count function nvc (the implementation of this function is provided in a later section). On line 21 the two other possible moves (up and left) are handled. An up move corresponds with inserting a gap in args1 (first sequence) and a left move corresponds with inserting a gap in args2 (second sequence). When adding gaps there is no gain in score (and neither is there a gap penalty in this implementation).

On line 25 the Enum.max_by function selects the move with the highest score. This result is then stored in the grid by putting it in the map at coordinate (x, y) on line 26.

After constructing the grid with optimal scores and moves, the algorithm enters the traceback phase to compute the actual alignment. On line 29, we start from the bottom-right corner of the grid $(n1, n2)$, and the traceback reconstructs the alignment by following the stored moves. If the move was diagonal (1, 1), it aligns the elements from both sequences.

For indices that are within the bounds of the grid we fetch the direction on line 35 using the previously computed grid. When the x- or y-index is out of bounds we must move towards the top-left $(-1, -1)$ position (lines 32 and 33). Once this final position is reached on line 31 the iterative "unfold" terminates due to a **nil** value being returned.

For each step in the path tracing, we obtain a pairwise alignment of terms and we compute an anti-unifier for each pair. This anti-unifier is returned as the first element of the tuple formed on lines 32, 33, 38, 40, and 42. The second element in these tuples is the new accumulator to use in the next iteration of the "unfold" operation. The result of the "unfold" is the result of the nw function and it is an anti-unifier for the aligned lists.

An example of the scoring matrix (i.e., scores in the dynamic programming table) and traceback is shown in Figure 4.1 for two string sequences of "TACGT" and "GACT". The best alignment path is determined by tracing back from the bottom-right corner to the top-left corner. This results in a pairwise alignment of "TG, AA, CC, G-, TT" where "-" is a gap. The key difference between this example and our implementation

Sequence 1

		T	A	C	G	T
	0	0	0	0	0	0
Sequence 2 G	0	0	0	0	0	0
A	0	0	1	1	1	1
C	0	0	1	2	2	2
T	0	0	1	2	2	3

Figure 4.1: Scoring matrix based on matches, mismatches, and gaps.

is that, instead of two characters (i.e., symbols) being compared, we need to handle comparing two arbitrary terms. This means the scoring function needs to be based on two aligned terms is the non-variable count of the LGG we compute for them.

4.2.3 Example

In this section we demonstrate an example of aligning two terms using Needleman–Wunsch. These two terms are both blocks with one having three elements and the other having four. In order to have an interesting generalization we will apply the higher-order form of anti-unification. The two (compact) terms as represented in Elixir are shown in Listing 4.5 and Listing 4.6.

Since there are three elements in the first term and four in the second we will have a three by four scoring matrix for the alignment. This is shown in Figure 4.2.

	T1 C1	T1 C2	T1 C3
T2 C1	-	-	-
T2 C2	-	-	-
T2 C3	-	-	-
T2 C4	-	-	-

Figure 4.2: Initial empty scoring matrix.

```
{:___block___, 3, [
  {:def, 2, [
    {:fo, 2, [
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args1, 0, []}]},
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args2, 0, []}]}
     ]},
    {:"[]", 1, [{:{}, 2, [:do,
        {:{}, 3, [
          {:f, 0, []},
          {:n, 0, []},
          {:zip_with, 3, [
            {:args1, 0, []},
            {:args2, 0, []},
            {:&, 1, [{:/, 2, [{:fo, 0, []}, 2]}]}
  ]}]}]}]}]},
  {:def, 2, [
    {:fo, 2, [{:x, 0, []}, {:x, 0, []}]},
    {:"[]", 1, [{:{}, 2, [:do, {:x, 0, []}]}]}
   ]},
  {:def, 2, [
    {:fo, 2, [{:x, 0, []}, {:y, 0, []}]},
    {:"[]", 1, [{:{}, 2, [:do, {:{}, 2, [{:x, 0, []}, {:y, 0, []}]}]}]}]}
   ]}
]}
```

Listing 4.5: First example term from source code of first-order anti-unification to explain alignment using Needleman–Wunsch.

```
{:___block___, 4, [
  {:def, 2, [
    {:ho, 2, [
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args1, 0, []}]},
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args2, 0, []}]}
    ]},
    {:"[]", 1, [{:{}, 2, [:do,
        {:{}, 3, [
          {:f, 0, []},
          {:n, 0, []},
          {:zip_with, 3, [
            {:args1, 0, []},
            {:args2, 0, []},
            {:&, 1, [{:/, 2, [{:ho, 0, []}, 2]}]}
  ]}]}]}]}]},
  {:def, 2, [
    {:ho, 2, [
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args1, 0, []}]},
      {:{}, 3, [{:g, 0, []}, {:n, 0, []}, {:args2, 0, []}]}
    ]},
    {:"[]", 1, [{:{}, 2, [:do,
        {:{}, 3, [
          {:{}, 2, [{:f, 0, []}, {:g, 0, []}]},
          {:n, 0, []},
          {:zip_with, 3, [
            {:args1, 0, []},
            {:args2, 0, []},
            {:&, 1, [{:/, 2, [{:ho, 0, []}, 2]}]}
  ]}]}]}]}]},
  {:def, 2, [
    {:ho, 2, [{:x, 0, []}, {:x, 0, []}]},
    {:"[]", 1, [{:{}, 2, [:do, {:x, 0, []}]}]}
  ]},
  {:def, 2, [
    {:ho, 2, [{:x, 0, []}, {:y, 0, []}]},
    {:"[]", 1, [{:{}, 2, [:do, {:{}, 2, [{:x, 0, []}, {:y, 0, []}]}]}]}
  ]}
]}
```

Listing 4.6: Second example term from source code of higher-order anti-unification to explain alignment using Needleman–Wunsch.

The algorithm proceeds from left to right and top to bottom filling in the cells with their scores. For the first cell the first clause in the first term is compared with the first clause in the second term. This means we first obtain a score using these two terms. To obtain this score we first compute their generalization. These two terms with their (higher-order) generalization are shown in Listing 4.7.

```
# Term 1
{:def, 2, [{:fo, 2, [
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args1, 0, []}]},
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args2, 0, []}]}
    ]},
   {:"[]", 1, [{:{}, 2, [:do,
         {:{}, 3, [{:f, 0, []},
            {:n, 0, []},
            {:zip_with, 3, [
               {:args1, 0, []},
               {:args2, 0, []},
               {:&, 1, [{:/, 2, [{:fo, 0, []}, 2]}]}
]}]}]}]}]}
# Term 2
{:def, 2, [{:ho, 2, [
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args1, 0, []}]},
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args2, 0, []}]}
    ]},
   {:"[]", 1, [
      {:{}, 2, [:do,
         {:{}, 3, [{:f, 0, []}, {:n, 0, []},
            {:zip_with, 3, [
               {:args1, 0, []},
               {:args2, 0, []},
               {:&, 1, [{:/, 2, [{:ho, 0, []}, 2]}]}
]}]}]}]}]}
# Generalization
{:def, 2, [
   {{:fo, :ho}, 2, [
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args1, 0, []}]},
      {:{}, 3, [{:f, 0, []}, {:n, 0, []}, {:args2, 0, []}]}
    ]},
   {:"[]", 1, [{:{}, 2, [:do, {:{}, 3, [
            {:f, 0, []}, {:n, 0, []},
            {:zip_with, 3, [
               {:args1, 0, []}, {:args2, 0, []},
               {:&, 1, [{:/, 2, [{{:fo, :ho}, 0, []}, 2]}]}
]}]}]}]}]}
```

Listing 4.7: First element from the first term and first element from the second term along with their higher-order generalization.

After obtaining a generalization we calculate a score using the non-variable count. For the first cell this score is 20. Since there are no cells to above and/or to the left, we set the cell's score simply to 20.

We then proceed to the next cell which compares the second clause in the first term with the first clause in the second term. The score is 4 for the generalization of these terms. However, for the second cell we also look to the cell to the left of it and determine the maximum value. Since 20 is larger than 4 we set the value of the second cell to 20. A similar situation occurs for the third cell. At this point the scoring matrix has the values as shown in Figure 4.3.

	T1 C1	T1 C2	T1 C3
T2 C1	20	20	20
T2 C2	-	-	-
T2 C3	-	-	-
T2 C4	-	-	-

Figure 4.3: Scoring matrix with first row filled in.

On the second row, in the first column we compare the first clause in the first term with the second clause in the second term. The generalization for these two terms has an NVC of 18 which is still less than the value in the cell above it (20). So this cell will again be set to 20. For the next cell, there is a value in the cell to the upper left of it, so the score of that cell is added to the NVC when determining the value. This results in 20+4, which is larger than both the value in the cell to the left or above, so the value set is 24. The scoring matrix at this point is shown in Figure 4.4.

	T1 C1	T1 C2	T1 C3
T2 C1	20	20	20
T2 C2	20	24	-
T2 C3	-	-	-
T2 C4	-	-	-

Figure 4.4: Scoring matrix with five cells filled in.

For the sixth cell, the NVC is 4 and we add 20 from the cell to the upper-left giving us 24 which corresponds with the score if the terms are aligned. Of the three values 24 is the maximum value so the sixth cell is set to 24. This process is repeated for the remaining cells resulting in a completed matrix as shown in Figure 4.5. After filling the dynamic programming matrix with scores, backtracing reconstructs the optimal alignment by following the path that led to the highest scores. Backtracing begins at the bottom-right corner of the matrix (the final cell), which contains the optimal alignment score for the complete sequences. At each cell during backtracing, we determine which previous cell contributed to the current cell's score by checking the three possibilities of vertical, horizontal, and diagonal. Note that there can be more than one valid option if there are multiple ways of aligning the sequences that result in the same optimal score.

	T1 C1	T1 C2	T1 C3
T2 C1	20	20	20
T2 C2	20	24	24
T2 C3	20	27	29
T2 C4	20	27	36

Figure 4.5: Scoring matrix with all cells filled in and the optimal path traced.

The resulting pairwise alignment is 'T1 C1' and 'T2 C1', gap and 'T2 C2', 'T1 C2' and 'T2 C3', and 'T1 C3' and 'T2 C4.' In Listing 4.8 the alignment result based on the source code the terms represented is shown. We see that the generalization of the first-order and higher-order anti-unification implementations captures a difference of an additional clause.

```
1  def fo({f, n, args1}, {f, n, args2}), do: {f, n, zip_with(args1, args2, &fo/2)}
2  def ho({f, n, args1}, {f, n, args2}), do: {f, n, zip_with(args1, args2, &ho/2)}
3
4  # GAP
5  def ho({f, n, args1}, {g, n, args2}), do: {{f,g}, n, zip_with(args1, args2, &ho/2)}
6
7  def fo(x, x), do: x
8  def ho(x, x), do: x
9
10 def fo(x, y), do: {x, y}
11 def ho(x, y), do: {x, y}
```

Listing 4.8: Resulting alignment of the two source code examples.

4.2.4 Equational theory

The alignment function is applied to block terms by adding a clause to the FO and HO implementations as shown in Listings 4.9 and 4.10. When a block term is encountered (line 1) the two list arguments are anti-unified using the Needleman–Wunsch alignment implementation from the previous section. As mentioned earlier, the third argument to the nw function is the respective anti-unification function to use within the nw function. This is denoted by &fo_et/2 for the fo_et function (the /2 refers to the function arity) and &ho_et/2 for the ho_et function.

One limitation of the implementation is that it omits the size of the block in the resulting term and instead denotes its absence with the value **nil**. Since the nw function computes the size of the block, the implementation could be modified so that the size is returned and inserted as the arity term. However, this was not done for the sake of simplicity and for the specific purposes of our research we do not use this value.

Other improvements to the implementation could include optimizing the clauses with pre-computed values for blocks of size one or zero. Depending on the frequency of block sizes it may also improve performance to use optimized code for individual cases such as small sequences with length two or three.

4.3 Non-variable count implementation

The Needleman–Wunsch implementation depends on calculating a score from an anti-unifier. We use the non-variable count (NVC) for this score and present its implementation in this section.

For both input terms and anti-unified terms the NVC is implemented the same as shown in Listing 4.11. On line 1, function terms with a variable function symbol are handled by mapping each argument recursively with nvc (denoted by &nvc\1) and summing together all the counts. Similarly, function terms with a non-

```
1 def fo_et({:__block__, _, args1}, {:__block__, _, args2}), do: {:__block__, nil, nw(args1, args2, &
        fo_et/2)}
2 def fo_et({f, n, args1}, {f, n, args2}), do: {f, n, Enum.zip_with(args1, args2, &fo_et/2)}
3 def fo_et(x, x), do: x
4 def fo_et(x, y), do: {x, y}
```

Listing 4.9: Elixir implementation of the first-order with equational theory form of anti-unification.

```
1 def ho_et({:__block__, _, args1}, {:__block__, _, args2}), do: {:__block__, nil, nw(args1, args2, &
        ho_et/2)}
2 def ho_et({f, n, args1}, {f, n, args2}), do: {f, n, Enum.zip_with(args1, args2, &ho_et/2)}
3 def ho_et({f, n, args1}, {g, n, args2}), do: {{f,g}, n, Enum.zip_with(args1, args2, &ho_et/2)}
4 def ho_et(x, x), do: x
5 def ho_et(x, y), do: {x, y}
```

Listing 4.10: Elixir implementation of the higher-order with equational theory form of anti-unification.

variable function symbol are calculated the same but with one added to the count due to the function symbol (line 2). On line 3, variable terms represented by two-tuples are given a count of zero. Lastly, any term not matching the previous ones is a non-variable and so is counted as 1 (line 4).

```
1 def nvc({{_, _}, _, args}), do: args |> Stream.map(&nvc/1) |> Enum.sum
2 def nvc({_, _, args}), do: 1 + (args |> Stream.map(&nvc/1) |> Enum.sum)
3 def nvc({_, _}), do: 0
4 def nvc(_), do: 1
```

Listing 4.11: Elixir implementation for the NVC, which is the number of non-variable symbols in a term.

4.4 Performance measuring implementation

This section discusses the implementation details of the performance measuring process. The setup of data, the sampling method, and the timing of the anti-unification function are explained.

4.4.1 Sampling

As discussed in the approach chapter, reducing variance in performance measurements is a necessary step to take by sampling multiple times. To determine the number of samples to take, we want to know after how many samples the measurements appear to stabilize. This can be done by taking a subsampling of the dataset, taking a large number of performance measurements repeatedly, then calculating the average change for the individual medians.

For the first-order form, we show this as a plot in Figure 4.6. Note that there are two lines in the plot, as the experiment was performed twice. Based on this we observe that variance is a significant concern, which justifies taking twenty-five samples to reduce the variance sufficiently while avoiding taking an unnecessarily large number of samples. Similar plots were obtained for the other anti-unification forms and so the choice of twenty-five samples was made for all forms in calculating the median performance timing for each individual pair of terms.

4.4.2 Setup

In the setup phase the source code strings are loaded, transformed, and stored in memory. Listing 4.12 shows the implementation which includes randomly sampling from all possible pairs and the use of the :persistent_term module to store the input terms. The significance of :persistent_term is related to memory optimization and is explained later. In lines 5–9, the data is loaded and encoded into terms using the string_to_compact_quoted function discussed in Section 4.1. We add an index for each term (line 8) and store

```
def setup() do
  num_samples = 10_000
  :rand.seed(:exsss, 0)

  data =
    get_source_strings()
    |> Stream.map(&string_to_compact_quoted/1)
    |> Stream.with_index()
    |> Enum.into(%{}, fn {x, i} -> {i, x} end)

  Enum.each(data, fn {k, term} ->
    :persistent_term.put({:term, k}, term)
  end)

  n = map_size(data)
  pairs = for a <- 0..n-1, b <- a+1..n-1//1, do: {a, b}
  samples = pairs |> Enum.shuffle() |> Enum.take(num_samples)

  {data, samples}
end
```

Listing 4.12: Elixir implementation of performance measuring setup.

```
def do_measurement_batch(samples, kind, seed, data) do
  :rand.seed(:exsss, seed)
  samples
  |> Enum.shuffle
  |> Enum.map(fn {id1, id2} ->
    timing = do_measurement([id1, id2], kind, data)
    %{ids: [id1, id2], timing: timing}
  end)
end
```

Listing 4.13: Elixir implementation of performance measurement batch.

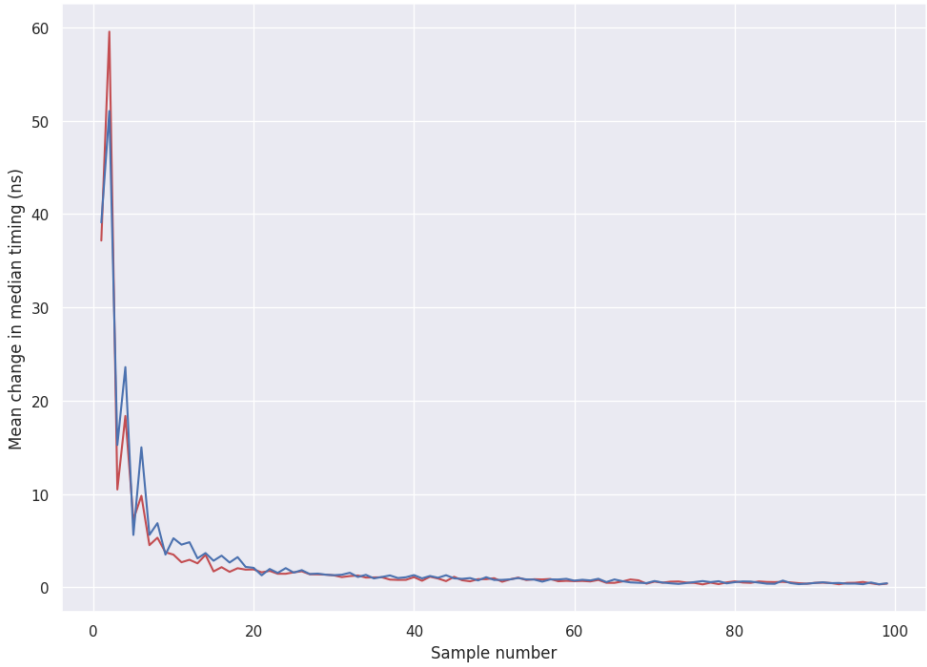

Figure 4.6: Line plot showing the change in median as more samples are taken. The experiment was repeated twice and so two lines are shown.

these in a map (line 9) with the index as the key and the term as the value. In line 12, each of the terms are stored in memory as "persistent" terms. In lines 15–17, the samples are drawn by shuffling a list of all possible pairs and taking ten thousand elements from this. On line 19 we return the indexed terms as a map and the list of sampled indices. These are used in the rest of the experiment process.

4.4.3 Measurement sample batching

To obtain twenty-five samples for each sampled pair we repeat a "measurement batch" (one measurement per term pair across all sampled terms) twenty-five times with the sequence of sampled pairs randomized each time. Listing 4.13 shows the implementation for a single batch. The list of samples (pairs of ids of the input terms) is first shuffled then for each pair a timing measurement is performed. A seed parameter is used on line 2 to make this shuffling (line 4) different each time.

The caller of this function (not shown) must pass the seed for the random number generator and the `kind` as an atom corresponding with the form of anti-unification to use (e.g., `:fo`). It also needs to handle repeating the batch and accumulating the timing results.

4.4.4 Memory access optimization

To implement the do_measurement function used in the do_measurement_batch function, there were three choices explored with varying implications on memory access. Across each of them, the timing method is the same (lines 3, 10, 20) with use of the tc function from the :timer Erlang standard library. This tc function takes a module (AntiUnification here), a function name (passed as the kind parameter here), and an argument list (args here). The key difference between the implementations is in the handling of these input arguments.

Inputs that are being anti-unified need to first be stored in the memory of the computer before being fetched by the CPU to get processed. Delays in memory access have a large impact on performance and so an appropriate strategy needs to be taken to avoid this issue. The three strategies attempted were (1) using a single Erlang process for all anti-unification calls, (2) using a new process for each anti-unification call with memory copying, and (3) using a new process for each anti-unification call without memory copying (by using :persistent_term).

The first and simplest choice is to simply call the anti-unification function using regular input terms: this is the v1 implementation as shown in Listing 4.14.

In the v2 implementation, a separate process is started using the Elixir Task module. The advantages of this are that separate processes have isolated memory and that a fresh process should have reduced execution pauses due to Erlang's preemptive scheduler. This implementation has, however, an implicit memory copy of the input arguments due to the memory isolation of processes.

Although the cost of the memory copy should occur before the timing of the anti-unification function starts, memory copying has an impact on the CPU's cache management. To avoid the memory copy, the v3 implementation uses the Erlang :persistent_term module which allows sharing the same term across different processes without the need for memory copying.

In Table 4.1, we show the timing statistics for these different methods. Note that while the mean for the "same process" version is lower than the "separate process" version, this is not true for their median timings. This suggests that for smaller sizes of data the "separate process" works well but as the data gets larger the timing substantially increases (due to memory copying overhead). With the "separate process and persistent_term" version the mean and median timings are lower than the other two versions with an improvement in the mean timing of 38%. This demonstrates the importance of memory access in the performance of anti-unification.

```
1 def do_measurement_v1(ids, kind, data) do
2   args = Enum.map(ids, &data[&1])
3   {timing, _} = :timer.tc(AntiUnification, kind, args, :
        nanosecond)
4   timing
5 end
6
7 def do_measurement_v2(ids, kind, data) do
8   args = Enum.map(ids, &data[&1])
9   (fn ->
10    {timing, _} = :timer.tc(AntiUnification, kind, args, :
        nanosecond)
11    timing
12  end)
13  |> Task.async()
14  |> Task.await(:infinity)
15 end
16
17 def do_measurement_v3(ids, kind, _) do
18  (fn ->
19    args = Enum.map(ids, &:persistent_term.get({:term, &1}))
20    {timing, _} = :timer.tc(AntiUnification, kind, args, :
        nanosecond)
21    timing
22  end)
23  |> Task.async()
24  |> Task.await(:infinity)
25 end
```

Listing 4.14: Elixir implementations of performance measurement.

Table 4.1: Table of summary statistics for performance timings in nanoseconds of implementations with different memory related effects.

Version	Mean	Std	Min	Median	Max
Same process	257	378	50	240	7,494
Separate process	422	1,577	50	201	32,020
Separate process and persistent_term	185	320	40	170	7,564

4.5 Input data

Our input dataset is obtained by taking the top hundred projects as ranked by libraries.io from Elixir's official package repository, Hex (hex.pm). Only projects with an explicitly permissive license are included. This approach is consistent with the approach of the CodeSearchNet dataset (Husain et al., 2019).

After obtaining the list of projects, the source code is obtained by downloading the latest version available through the package manager and extracting the files under the lib directory with the Elixir file extension of ".ex"; this ensures that test files are not included. We filter out empty files; we parse the remaining files via Elixir's standard library; we encode them in the compact quoted expression form, producing the dataset.

In Table 4.2, we show the summary statistics for our dataset of terms used as input to the experiment, including the number of terms (twice the number of pairs) and that all terms have at least one non-variable symbol. Since all terms are ground terms, the term non-variable count (NVC) gives the size of the inputs; thus we have a median size of 288 while the largest size is 18,440.

Table 4.2: Table of summary statistics for the input dataset of terms.

Attribute	N	Min	Median	Max	StDev
Term non-variable count (NVC)	20,000	1	288	18,440	1,288

4.6 Summary

This chapter discussed the implementation of encoding source code as terms, the implementation of the four anti-unification forms including the Needleman–Wunsch algorithm, and the non-variable count and performance measures. The impact of memory access on the performance was demonstrated by comparing implementations differing in how the tasks that perform anti-unification are spawned.

Chapter 5

Results and Analysis

In this chapter, we present the results for the performance measurements obtained for each of the trials. We model the performance of each form of anti-unification is modelled and we calculate effectiveness scores, with a gain in the commonalities between inputs treated as the objective. We analyze the differences in effectiveness between forms, both overall and at a fine granularity.

Section 5.1 describes the resulting data that we collected. Section 5.2 attempts to model this data, analyzing the nature of the results in the process. We compare the efficiency of the different forms of AU in Section 5.3. We summarize in Section 5.4.

5.1 Performance data

As a measure to filter out extreme outliers, we filter from the performance dataset all terms that have overall median timings above the 99.9th percentile. Consequently, the number of input pairs in this dataset is 9,990 rather than 10,000.

We show summary statistics in Table 5.1 for each of the four forms of anti-unification. Although FO and HO may obtain LGGs with an NVC of zero, for FO+ET and HO+ET there is always at least one non-variable symbol. Both the median and maximum values for the NVCs are approximately three to four times greater with the equational theory versus without it. Differences between the median execution times for forms with equational theory are about a thousand to two thousand times larger versus without it. In the absolute worst cases we have maximum times of about fifty milliseconds for the forms with equational theory. Without it, the slowest execution times are about 7.5 and 15 microseconds.

In Table 5.2, efficiency and effectiveness scores are shown for each form of anti-unification based on a gain in non-variable count for LGGs per unit of compute time. This score shows that the first-order form has the

Table 5.1: Results for the anti-unification forms of first-order (FO), higher-order (HO), first-order with equational theory (FO+ET), and higher-order with equational theory (HO+ET). The execution times here are based on the median of 25 samples taken for each pair of terms.

		LGG NVC			Execution time (ns)		
Trial version	N	Min	Median	Max	Min	Median	Max
FO	9,990	0	5	103	40	170	7,564
HO	9,990	0	5	103	50	171	14,998
FO+ET	9,990	1	18	358	1,493	269,513	47,181,610
HO+ET	9,990	1	22	391	1,673	405,272	68,107,710

Table 5.2: Absolute totals of LGG NVCs and time costs for data with timings less than the 99.9th percentile. The time cost totals are based on the median times. Effectiveness is compared against FO as the baseline to give a percentage increase.

Form	LGG NVC total	Time cost total	Efficiency	Effectiveness
FO	46,911	1,851 μs	25.3 MHz	100%
HO	48,558	2,234 μs	21.7 MHz	104%
FO+ET	276,901	9.378 ms	29.5 kHz	590%
HO+ET	329,883	14,689 ms	22.5 kHz	703%

highest efficiency and with the addition of the equational theory this efficiency is reduced by a thousand-fold. However, with the equational theory, the effectiveness is about 600% when compared to first-order as a baseline. The difference in total time cost shows that, without equational theory, all approximately ten thousand term pairs are anti-unified in about 2 milliseconds while adding the equational theory leads to total time costs of about 10 and 15 seconds for FO+ET and HO+ET, respectively.

In Figure 5.1, the timings for each of the forms of anti-unification is shown in a single summary plot of the logarithm of the median execution time (measured in nanoseconds) versus the term non-variable count for the LGG; each plotted dot represents a single case of pairwise anti-unification, where the colour signifies its form according to the legend shown. We see that the two forms without equational theory are clustered loosely towards the lower ends of both the x- and y-axes, while the forms with equational theory reside in a broader band and mostly at a greater execution time. This plot shows that there is minimal difference between FO and HO as well as between FO+ET and HO+ET. The addition of the equational theory increases the timing by multiple orders of magnitude. The plot also shows that for FO and HO forms the maximum LGG NVC is limited to about 50 except for a few outliers in the 100 range. In contrast, FO+ET and HO+ET have many LGGs with NVCs higher than 50 with values extending gradually up to around 300. Thus, the inclusion of equational theory decreases efficiency but increases effectiveness.

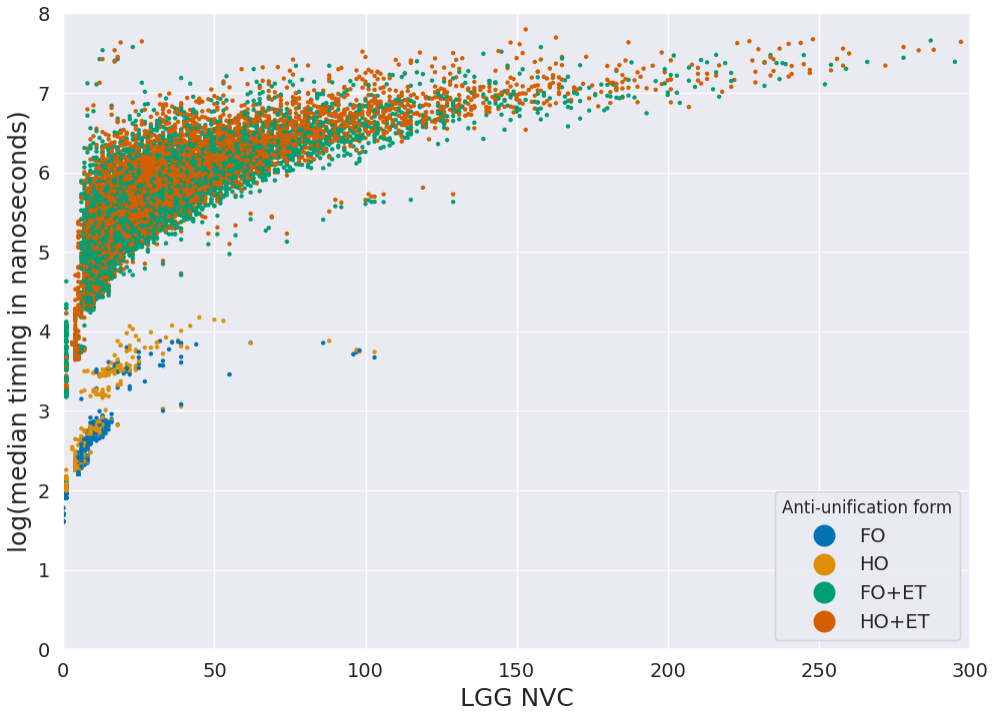

Figure 5.1: Scatter plot showing the log (base 10) of median timing across each of the four forms of anti-unification for each pair of terms.

5.1.1 Analysis of block symbols

The equational theory for the __block__ symbol will have varying impacts on the anti-unification process depending on the frequency of the symbol's occurrence in the input terms. In Table 5.3 we show the summary statistics of the __block__ symbol's frequency in the input terms as well as for the different anti-unification forms. For the input terms, every term begins with a __block__ symbol so all input terms will have at least one. This is also reflected in the frequencies for FO+ET and HO+ET forms where the minimum is one. We notice the biggest difference in the maximums where FO+ET and HO+ET compute anti-unifiers with 19 and 27 __block__ symbols respectively compared to FO and HO which have at most 5.

Table 5.3: Minimum, median, maximum, and standard deviation (SD) statistics for the frequencies of the __block__ symbol in input terms and the anti-unifier terms produced by each form of anti-unification.

Version	Min	Median	Max	SD
Input terms	1	4	199	12.70
FO anti-unifier	0	1	5	0.380
HO anti-unifier	0	1	5	0.382
FO+ET anti-unifier	1	2	19	1.33
HO+ET anti-unifier	1	2	27	1.64

5.1.2 Analysis of block arities

Our equational theory for the __block__ symbol impacts the occurrence of blocks in the anti-unifier. In particular, when the arities of blocks differ, the equational theory allows these to be generalized and included in the anti-unifier. Of the sampled pairs 96.41% have blocks of the same arity and in 94.93% of cases alignment was a problem.

To analyze the effect of the equational theory we can compare the frequencies of block arities between the input terms and the anti-unifier term when using different forms of anti-unification. Since blocks can themselves contain blocks, each term has its own frequency of block arities. Consequently, what we need to analyze is the distribution of those frequencies. In Table 5.4 we show the top ten occurrences of block arity frequencies for the input terms and in Table 5.5 for the various anti-unifiers. Without the equational theory we can see the frequencies of anti-unifiers with no blocks or only a single block of arity one are very high (8490 and 1294 for FO). With equational theory the most frequent case is still a single block of arity one but the frequencies are only 524 for FO+ET and 443 for HO+ET. For non-trivial arities we see that the most frequent arities are typically larger with equational theory $(14, 8, 5, 10)$ compared to without $(5, 3, 8, 6)$.

Table 5.4: Frequencies for top ten occurrences of block arity frequencies for input terms being anti-unified. Under block arities the frequency of block arities in a term is given in a comma delimited list of arity-frequency colon separated pairs. The frequency column provides the number of terms that for the corresponding block arity frequency.

Terms	**Rank**	**Block Arities**	**Frequency**
First input	1	1: 1, 5: 1	242
	2	1: 1, 2: 1	240
	3	1: 1, 3: 1	229
	4	1: 1, 4: 1	218
	5	1: 1, 6: 1	192
	6	1: 1, 8: 1	165
	7	1: 1, 7: 1	146
	8	1: 1, 2: 1, 4: 1	99
	9	1: 1, 2: 1, 5: 1	85
	10	1: 1, 11: 1	83
Second input	1	1: 1, 5: 1	478
	2	1: 1, 3: 1	432
	3	1: 1, 8: 1	381
	4	1: 1, 6: 1	232
	5	1: 1, 4: 1	193
	6	1: 1, 2: 1	154
	7	1: 1, 11: 1	141
	8	1: 1, 7: 1,	139
	9	1: 1, 2: 2, 3: 1, 5: 1	125
	10	1: 1, 9: 1	110

Table 5.5: Frequencies for top ten occurrences of block arity frequencies for anti-unifier terms of various forms of anti-unification (first-order, higher-order, first-order with equational theory, higher-order with equational theory). Under block arities the frequency of block arities in a term is given in a comma delimited list of arity-frequency colon separated pairs. The frequency column provides the number of terms that for the corresponding block arity frequency.

Terms	**Rank**	**Block Arities**	**Frequency**
FO	1	1: 1	8490
	2	-	1294
	3	1: 1, 5: 1	51
	4	1: 1, 3: 1	20
	5	1: 1, 8: 1	19
	6	1: 1, 6: 1	16
	7	1: 1, 4: 1	14
	8	2: 1	11
	9	1: 1, 7: 1	10
	10	1: 1, 13: 1	9
HO	1	1: 1	8488
	2	-	1294
	3	1: 1, 5: 1	51
	4	1: 1, 3: 1	20
	5	1: 1, 8: 1	19
	6	1: 1, 6: 1	16
	7	1: 1, 4: 1	14
	8	2: 1	11
	9	1: 1, 13: 1	10
	10	1: 1, 7: 1	9
FO+ET	1	1: 1	524
	2	1: 1, 14: 1	227
	3	1: 1, 8: 1	202
	4	1: 1, 5: 1	190
	5	1: 1, 10: 1	186
	6	1: 1, 7: 1	168
	7	1: 1, 9: 1	161
	8	1: 1, 12: 1,	158
	9	1: 1, 13: 1	157
	10	1: 1, 18: 1	155
HO+ET	1	1: 1	443
	2	1: 1, 14: 1	215
	3	1: 1, 8: 1	189
	4	1: 1, 10: 1	174
	5	1: 1, 5: 1	171
	6	1: 1, 9: 1	156
	7	1: 1, 18: 1	156
	8	1: 1, 12: 1,	154
	9	1: 1, 7: 1	152
	10	1: 1, 11: 1	150

5.2 Performance models

In this section, we model the execution time of each form of anti-unification included in the experiment. As a first step, we use Pearson correlation coefficients to determine which measure is most suitable as our independent variable for modelling the execution time (the dependent variable). This turns out to be the LGG NVC rather than the input term NVCs (i.e., sizes of inputs). Then we determine whether to use a linear or power model with the aid of a log–log plot on data with a non-zero NVC.

Unfortunately, the models we obtain are limited in their capacity to explain the variance in execution times. The potential causes of this limitation are discussed in Chapter 6.

5.2.1 Correlation coefficients

For FO and HO, the correlation coefficient between the sizes of inputs and the execution time is nearly zero. This means that for FO and HO the execution time does not appear to be dependent on the size of inputs. The NVC of the LGG, however, shows a strong correlation with execution time having a coefficient of approximately 0.8 in the case of both FO and HO. For FO+ET and HO+ET there is some correlation between the sizes of inputs and the execution time with coefficients of approximately 0.5. However, there is a much better correlation for the LGG NVCs with coefficients of approximately 0.75. Therefore, we conclude that, for each of the forms of anti-unification, the NVC of the LGG is the most appropriate measure, out of those we examined, for modelling execution time.

5.2.2 First-order model

Using the non-variable count of the LGG to model the performance of the first-order form did not result in a model that could appropriately account for more than approximately fifty percent of variance. While some of the data does fit an approximately linear model, the variance in the remaining data suggests additional factors are needed to explain this variance. One proposed explanation which will be discussed in Chapter 6 is the effect of the physical size of the symbol representations.

Figure 5.2 shows an attempt at estimating the exponent of a power model using a log–log regression plot. An R^2 value of 0.67 suggests that some data could fit a power model with an exponent of 0.70 but a lot of variance remains unexplained. The fit of this model on the original (i.e., no log–log transform) data has a very low R^2 value of 0.23. So we consider a linear model instead.

Figure 5.3 shows an attempt at fitting the (original) data using linear regression which results in a higher R^2 value of 0.56.

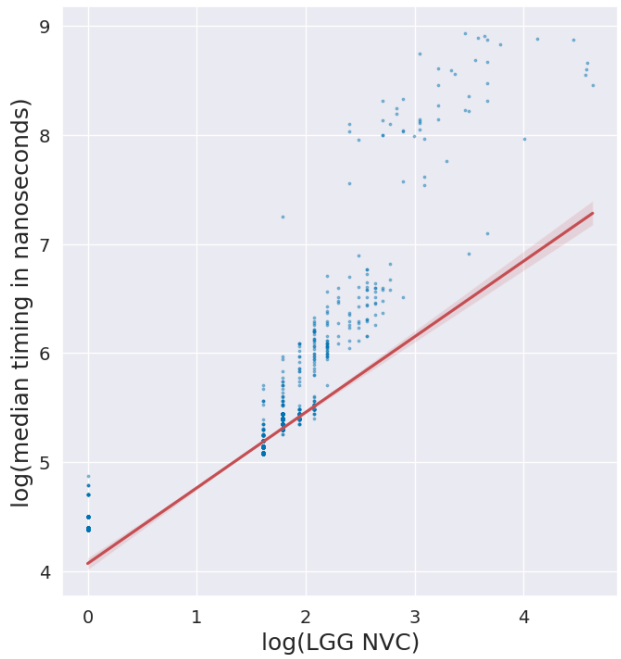

Figure 5.2: For the FO form, using linear regression on the transformed data we have an R^2 value of 0.67 and it gives a final equation of $\log(y) = 0.70 \cdot \log(x) + 4.1$.

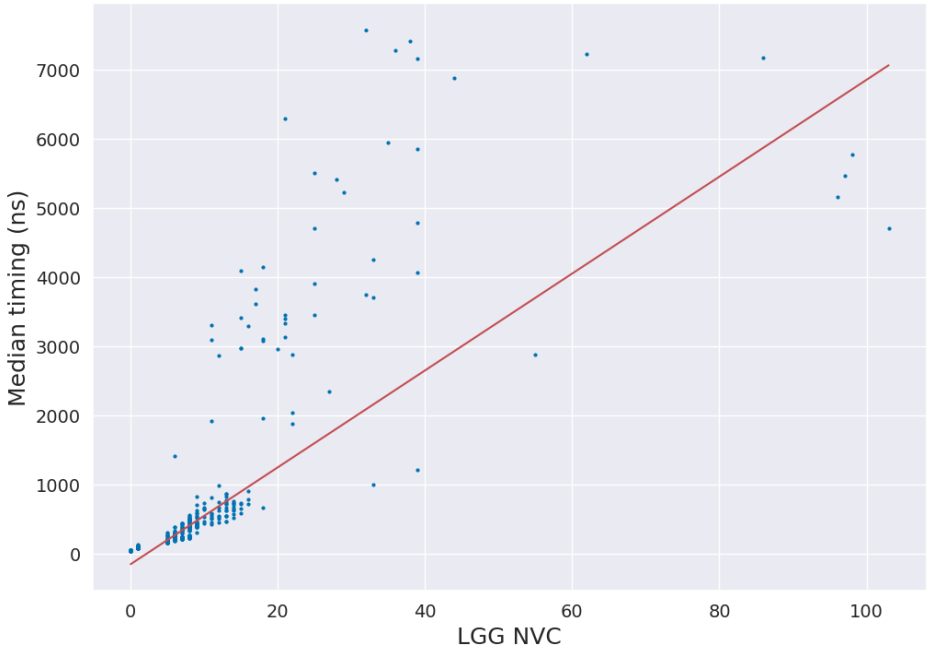

Figure 5.3: For the FO form, using linear regression on the untransformed data we have an R^2 value of 0.56 with an equation of $y = 70x - 150$.

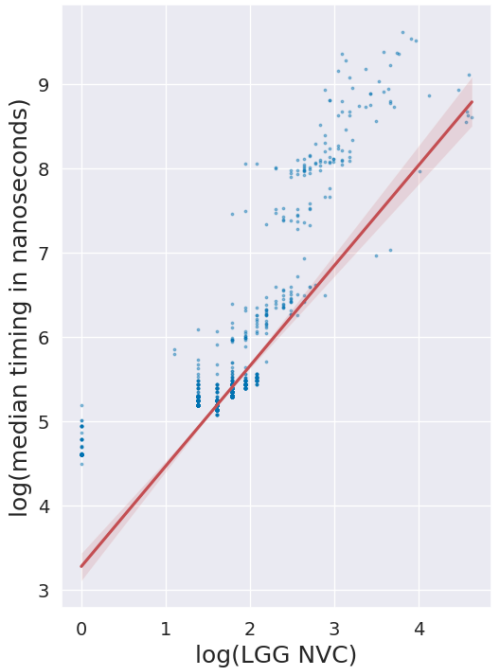

Figure 5.4: For the HO form, using linear regression on the log transformed data we have an R^2 value of 0.63 and it gives a final equation of $\log(y) = 1.19 \cdot \log(x) + 3.3$.

Figure 5.5: For the HO form, using linear regression on the untransformed data we have an R^2 value of 0.50 with an equation of $y = 110x - 330$.

5.2.3 Higher-order model

The performance of the higher-order form has many similarities with the first-order form; however, the effects of the additional cost of forming variables from the function symbols can be seen in the data. This again highlights the limitation of the non-variable count in modelling the performance as the number of variables in the result should have an effect on the compute cost but the NVC will not reflect this.

Figure 5.4 shows an attempt at estimating the exponent of a power model using a log–log regression plot. An R^2 value of 0.63 suggests that some data could fit a power model with an exponent of 1.19 but a lot of variance remains unexplained. Using the coefficient and exponent from the log–log linear regression fit results in an R^2 value of 0.37 on the untransformed data.

Figure 5.5 shows an attempt at fitting the data using linear regression. This results in a better R^2 value of 0.50 compared to the fit from the log–log regression.

5.2.4 First-order with equational theory

The performance of the forms with equational theory has quite different characteristics from the FO and HO forms.

Figure 5.6 shows an attempt at estimating the exponent of a power model using a log–log regression plot. The resulting equation of $y = 2500x^{1.6}$ has an R^2 value of 0.60 on the untransformed data; its plot is shown in Figure 5.7.

5.2.5 Higher-order with equational theory

The performance of the HO+ET form is extremely similar to FO+ET; however, the timings are generally higher and the exponent estimated with the log–log regression is slightly higher.

Figure 5.8 shows an attempt at estimating the exponent of a power model using a log–log regression plot which has fairly good fit with an R^2 value of 0.80. The resulting equation of $y = 1600x^{1.8}$ has an R^2 value of 0.64 on the untransformed data; its plot is shown in Figure 5.9.

5.3 Efficiency Comparisons

The overall efficiency computed across the entire dataset was given in a previous section. In this section we analyze how differences in efficiency between FO and HO and between with and without ET vary across the dataset.

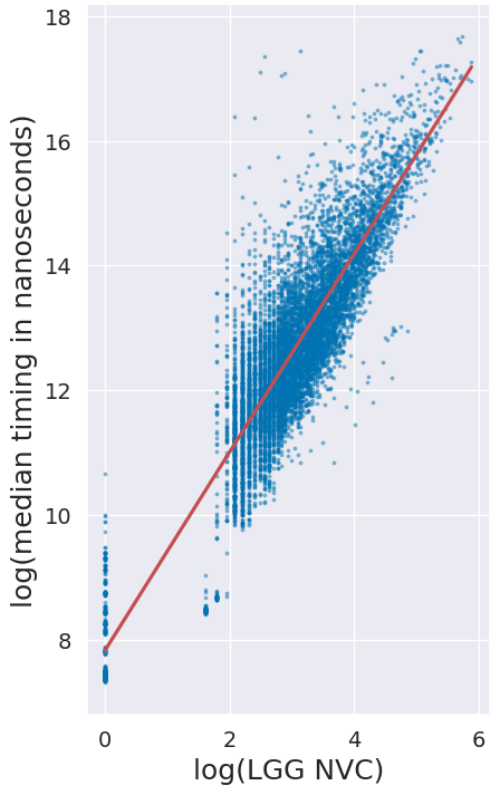

Figure 5.6: For the FO+ET form, using linear regression on the log transformed data we have an R^2 value of 0.82 and a corresponding equation of $\log(y) = 1.6 \cdot \log(x) + 7.8$.

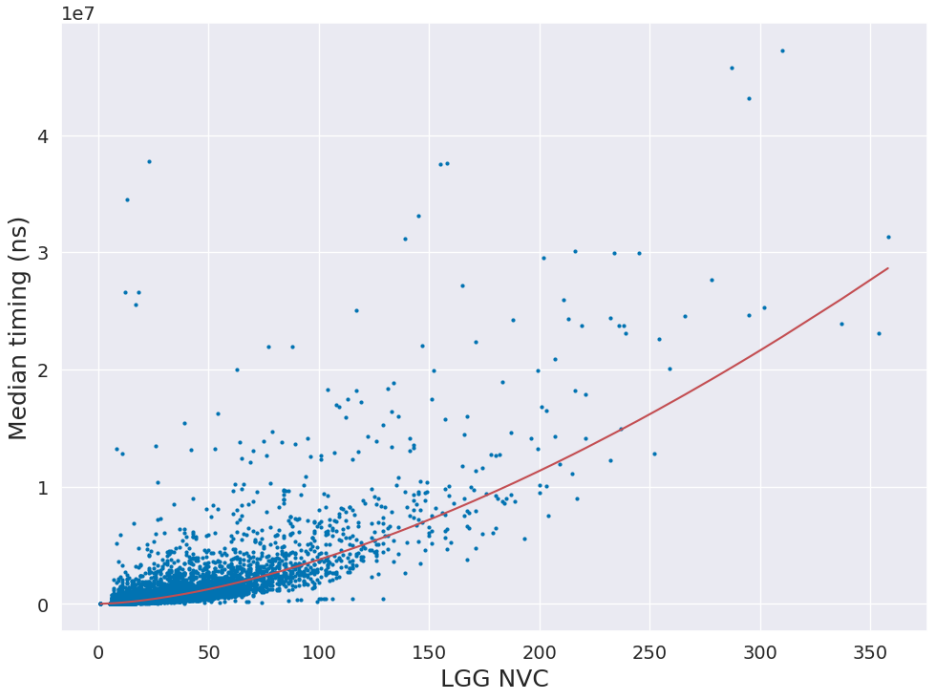

Figure 5.7: For the FO+ET form, an R^2 value of 0.60 is obtained using the model $y = 2500x^{1.6}$.

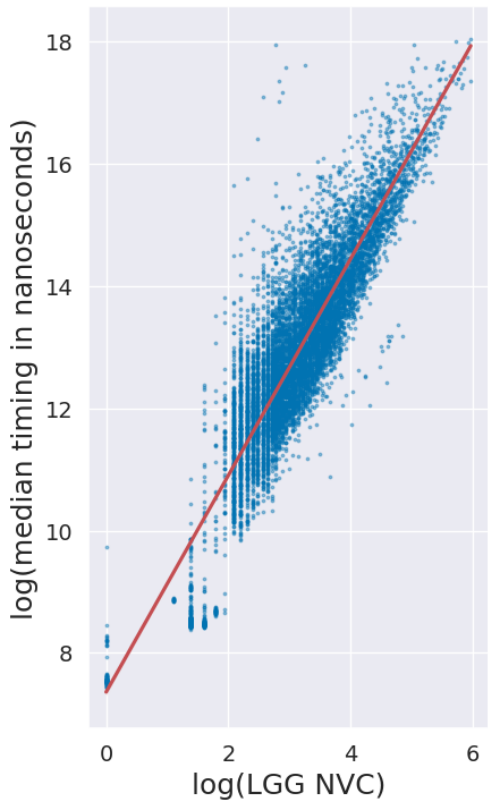

Figure 5.8: For the HO+ET form, using linear regression on the log transformed data we have an R^2 value of 0.80 and a corresponding equation of $\log(y) = 1.8 \cdot \log(x) + 7.4$.

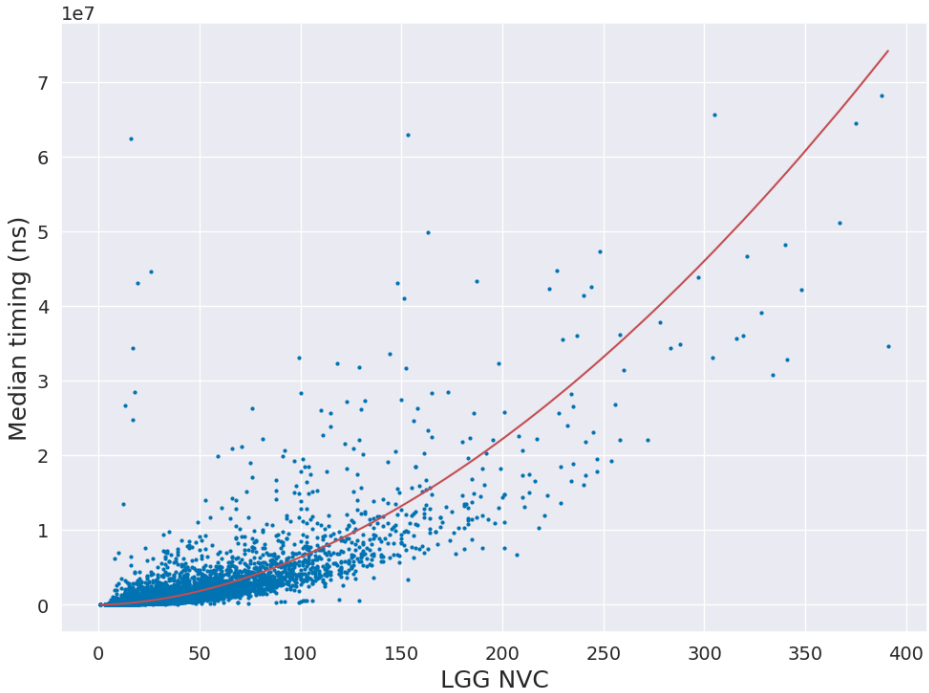

Figure 5.9: For the HO+ET form, an R^2 value of 0.64 is obtained using the model $y = 1600x^{1.8}$.

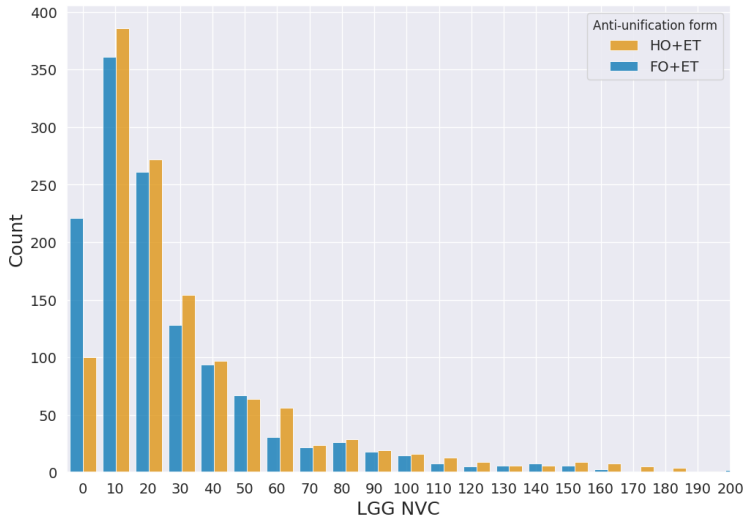

Figure 5.10: Histogram of NVC for the LGG of FO+ET and HO+ET when FO LGG NVC is zero.

An interesting characteristic of the forms of anti-unification studied in this thesis is that the HO/HO+ET forms always find more commonalities over the FO/FO+ET forms and the FO+ET/HO+ET always find more commonalities over the FO/HO forms.

The first-order form is unique among the four forms in having no correlation between the NVC of its LGG and its efficiency when excluding LGGs with an NVC of zero. This suggests that the FO form tends to find the same total number of commonalities per unit of cost regardless of whether each pair of terms has many or few commonalities. In other words, if there are few commonalities then the cost is lower and if there are many then the cost is proportionately higher.

On the other hand, the other forms have negative correlations between the LGG NVC and efficiency. So for these cases we will expect a larger per unit cost when the pair of terms have fewer commonalities. With more commonalities, the efficiency improves.

Based on this we should expect the difference in efficiency between forms for individual term pairs to have significant variance. To analyze this, we calculate a ratio between the efficiency of one form versus another form and plot these values using a histogram. The equation for this is efficiency ratio = $\text{efficiency}_1/\text{efficiency}_2$. Since the efficiency can be zero when the NVC is zero we exclude such data and analyze it separately.

In Figure 5.10, the NVCs for FO+ET and HO+ET are shown for the subset of data where the FO LGG has an NVC of zero. For this data, it is also the case that the NVC for the HO LGG is also zero whenever the FO LGG has an NVC of zero, so the HO form is omitted from the plot. Based on this histogram we see that in the majority of cases the equational theory finds some commonalities even if the FO form has found none.

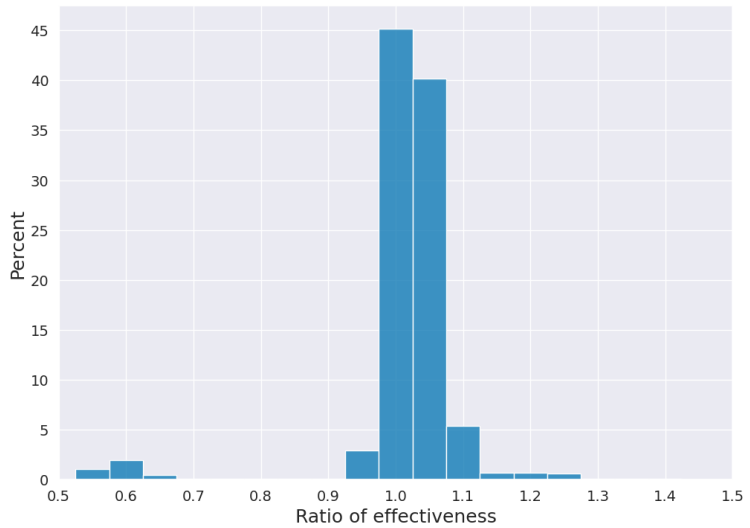

Figure 5.11: Histogram of ratios of FO efficiency to HO efficiency. Approximately 45% of the data has an approximately equal ratio with FO efficiency being 1.0× the efficiency of HO.

The next subsections examine the histograms for the efficiency ratio based on the data where the FO form has an LGG with a non-zero NVC. Since each of the other forms cannot have a lower NVC, we know that, for any efficiency ratio we calculate, there will not occur any zeroes in either of the efficiency measures it is based on.

5.3.1 First-order versus higher-order

We use two histograms to visualize the efficiency ratios for FO to HO and for FO+ET to HO+ET. Figure 5.11 shows the ratio of FO efficiency to HO efficiency, while Figure 5.12 shows the ratio of FO+ET efficiency to HO+ET efficiency. For most instances the ratio is between 1.0 and 1.1, showing that FO generally has better efficiency with respect to HO. However, in rare cases the efficiency is higher for HO compared to FO with some ratios being measured around 0.6 or 0.9.

Although the overall efficiency is higher for FO versus HO and for FO+ET versus HO+ET, we see that this is not true for every individual instance. We find that for approximately five percent of cases the HO/HO+ET form has a better efficiency. These cases will be discussed in Chapter 6.

Additionally, the histograms show that without the equational theory, the FO and HO forms typically have a minimal difference in efficiency. However, when combining with the equational theory, the difference becomes more pronounced with a typical decrease in efficiency of twenty percent due to the use of HO instead of FO.

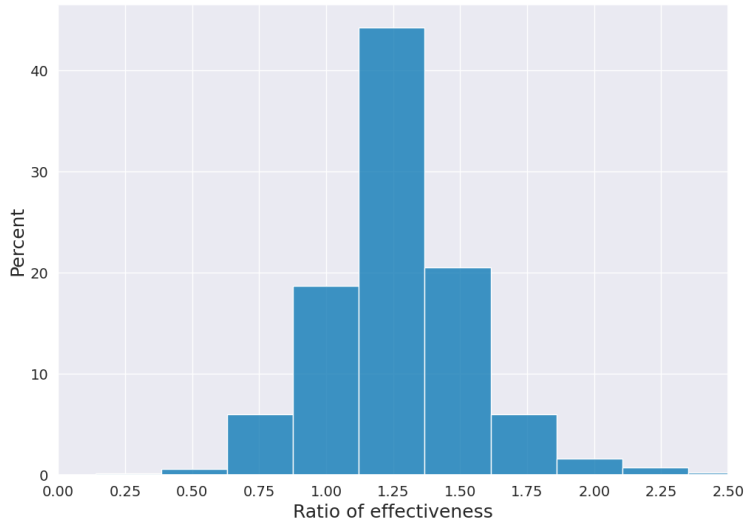

Figure 5.12: Histogram of ratios of FO+ET efficiency to HO+ET efficiency. For approximately 45% of the data FO+ET efficiency is 1.25× the efficiency of HO+ET.

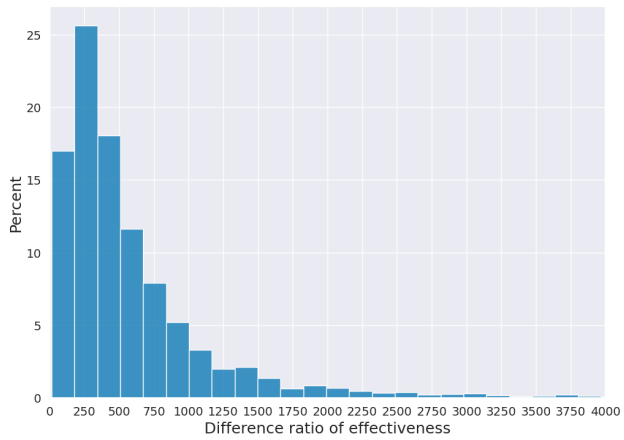

Figure 5.13: Histogram of ratios of FO efficiency to FO+ET efficiency. The median ratio puts FO efficiency at 400× that of FO+ET.

5.3.2 Without versus with equational theory

We use two histograms to visualize the efficiency ratios for FO to FO+ET and for HO to HO+ET. Figure 5.13 shows the ratio of FO efficiency to FO+ET efficiency, while Figure 5.14 shows the ratio of HO efficiency to HO+ET efficiency. Note that these plots are not able to show the minimum ratios effectively: 14.2× and 13.1× for the ratios of FO to FO+ET and of HO to HO+ET respectively.

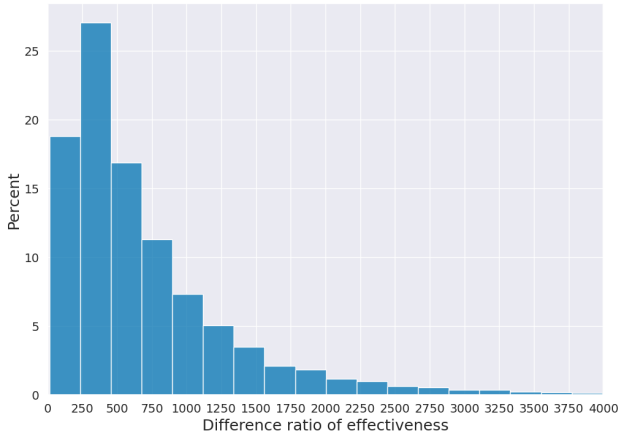

Figure 5.14: Histogram of ratios of HO efficiency to HO+ET efficiency. The median ratio puts HO efficiency at 500× that of HO+ET.

5.4 Summary

In this chapter we obtained performance models for each form of anti-unification being studied and compared the efficiency and effectiveness between them.

Using Pearson correlation coefficients we determined that the NVC of LGG is a good choice for modelling execution time. Linear models resulted in better fits compared to power models for both FO and HO forms whereas power models resulted in better fits for FO+ET and HO+ET. Unfortunately, the R^2 values were relatively low meaning at most about sixty percent of the variance in performance timings could be explained. This limitation arising from the limited predictive power of the LGG NVC is discussed in the next chapter.

The differences in efficiency between the forms were calculated and analyzed in fine detail. While the forms with equational theory were about a thousand times less efficient overall, the median ratios on individual inputs were only 400× and 500× less for FO+ET versus FO and HO+ET versus HO respectively. Overall, HO was less efficient compared to FO, but in about five percent of cases the inverse is true with HO being about thirty to seventy percent more efficient.

Differences in effectiveness were also calculated and analyzed. With the FO form as a baseline (that is, by giving the FO form an effectiveness of 100%), HO has a similar effectiveness of 104%, while FO+ET and HO+ET have higher effectiveness with values of 590% and 703% respectively.

Chapter 6

Discussion

The different forms of anti-unification studied in this thesis are different formal problems. But, as evidenced by the performance results, FO and HO have similar time complexity whereas the addition of equational theory results in a much harder problem, at least for the specific equational theory that we have used.

Therefore, the goal of comparing effectiveness and efficiency between the forms with versus without equational theory is not to determine which performs "better" in absolute terms but to quantify the trade-offs involved.

The three key results obtained in this thesis are that (a) semantic generalization is more effective than syntactic generalization; (b) semantic generalization is less efficient than syntactic generalization; and, (c) the quantified trade-off between effectiveness and efficiency has some generalizability between different forms.

In Section 6.1 the effectiveness is addressed, in Section 6.2 the efficiency is addressed, and in Section 6.3 the trade-offs are addressed.

Generally we expect lower order forms to overall outperform higher order ones. Likewise for equational theories, we can generally expect that, when comparing on the same hardware, the form without equational theory overall outperforms the forms with equational theory. However, these expectations might not necessarily be accurate across different situations. We discuss limitations in Section 6.4 and potential threats to validity in Section 6.5.

Finally, we summarize in Section 6.6.

6.1 Effectiveness

The results show that using a higher-order form versus a first-order form does provide effectiveness gains in the sense that more commonalities are obtained. The same result was obtained for the forms with

our equational theory versus without. However, the difference in effectiveness for the equational theory is much higher compared to the difference from higher-order. Additionally, when combining higher-order and equational theory, we find that the effectiveness is maximized and that the impact of using a higher-order form is greater than for the forms without equational theory.

Another important finding is that the effectiveness (or lack of effectiveness) of syntactic generalization (i.e., without equational theory) is not predictive of the effectiveness of semantic generalization (i.e., with equational theory). This is reasonable to expect since the very purpose of semantic generalization is to compare terms that are not syntactically similar. This demonstrates that, for the dataset we studied, the syntax and semantics are to some extent statistically independent.

Overall our results support the claim that semantic generalization is more effective than syntactic generalization.

6.2 Efficiency

The improvements in effectiveness are unfortunately counter-balanced by large drops in efficiency. Importantly, the loss of efficiency is proportional to the improvement in effectiveness. However, this is not a linear relationship. In the case of equational theory, for an approximately seven times increase in effectiveness, we find an approximately thousand times decrease in efficiency. Likewise for the higher-order forms we find losses in efficiency along with the gains in effectiveness albeit of smaller magnitudes.

Overall our results support the claim that semantic generalization is less efficient than syntactic generalization.

6.2.1 Impact of equational theory on efficiency

The FO and HO forms of anti-unification in this thesis are solvable by simply abstracting at any point (as allowed by the particular form) where there is an absence of commonality between inputs. Consequently, the relationship between an input term and its LGG computed by FO or HO is that, wherever a variable term occurs in the LGG, it generalizes a more specific term within the input term.[1]

In contrast, the addition of equational theory defines a harder problem, where the relationship between an input term and an LGG is less direct. Because terms that differ syntactically can now be considered equivalent, the instantiation of the LGG does not necessarily need to produce a term syntactically identical to an input term, so solving the problem of anti-unification modulo equational theory not only finds a

[1]This is true for the input terms considered in this thesis but does not hold in the more general case when input terms are not necessarily ground terms.

set of commonalities (which can be be quantified by the NVC) but also mappings for each input term to an equivalent one instantiable from an LGG. For the blocks equation studied in this thesis, this mapping corresponds with a pairwise sequence alignment.

This idea generalizes to other equational theories when the same situation occurs with multiple solutions being compared to a unique (modulo renaming) solution obtained from first-order anti-unification. In the case of our blocks equation we used pairwise sequence alignment to compute every possible solution but this may not work for other equational theories that constitute undecidable problems.

Returning to the effectiveness measure calculated using a gain in commonalities we can see that this is missing any gain from obtaining the alignment or more generally from finding multiple solutions. In measuring the effectiveness of FO/HO this makes sense since these forms ignore the problem of structural alignment and we find only one unique solution. Therefore, the loss of effectiveness in FO+ET/HO+ET compared to FO/HO is just an aspect of a trade-off that is potentially balanced by an additional gain not available from the FO/HO forms.

In principle, the alignment gained could be quantified is some manner, but it is not obvious how this should be done. On the other hand, the cost incurred from computing the alignment can be calculated by comparing against the effectiveness obtainable without any alignment (i.e., FO/HO). Based on this interpretation, the cost of alignments (more generally the cost of the equational theory) ranges from $10\times$ to $80,000\times$ with $400\times$ being the median. This provides a quantification of the relative difficulty of generalization with equational theory versus without. Additionally, the distribution of difficulties for the individual generalizations within the dataset is quantifiable. This has a potential application in dividing the dataset based upon the difficulty of generalization. For the more difficult cases an approximation likely would have a greater potential to provide larger gains in effectiveness.

Another application would be in the creation of a benchmark of difficult generalization problems. Because an arbitrary pair of terms may not necessarily have a "good" generalization, these difficult cases can also be used to evaluate the effectiveness of methods for classifying whether a given pair of terms will result in a non-trivial generalization. This is recognized as an important problem in the ASGard tool (Mahmoud et al., 2024) where a clustering method is used to group terms together before applying generalization.

Having a set of difficult example cases (derived from real-world data) is important for evaluation because these are more representative of the cases where a software engineer would benefit from using an automated tool. If a generalization problem is too easy, then the additional effort of reaching for and using a tool may not be justified. From these difficult cases, there could also be a manual selection of cases that are not syntactically similar but can be generalized with an appropriate equational theory. Having real-world examples of generalization that depend on a sufficiently advanced equational theory would be valuable to

performing an empirical evaluation that quantifies the impact of different equational theories. This knowledge of which equational theories are most valuable in practice is essentially metaknowledge of what software engineers typically need to know to accomplish generalization tasks — information that would generally be invaluable to designers of tools.

6.3 Trade-offs and generalizability

Because we find that effectiveness and efficiency cannot be improved simultaneously, the trade-offs between different forms are of particular importance. A key question is how well these generalize so we can predict the trade-offs for other forms. We find that the trade-offs between FO and HO are generally predictive of the trade-offs between FO+ET and HO+ET but not entirely accurate in terms of magnitude: whereas the efficiency loss and effectiveness gain for FO versus HO are fairly small, the trade-off for FO+ET versus HO+ET is much larger.

There are two important conclusions that can be drawn from analyzing these trade-offs. Firstly, we can give a partial answer to when semantic generalization should be used. Because we must consider the trade-offs, and the effectiveness of syntactic generalization is not predictive of the effectiveness of semantic generalization, then it makes sense to preferentially use semantic generalization in cases where syntactic generalization is not effective. This is because there is more potential upside from applying semantic generalization in those cases where we do not already have a good result from applying syntactic generalization. Secondly, if we are using more than one form of generalization (e.g., to obtain a profile of the various trade-offs) then it is preferable from an efficiency perspective to use HO and FO+ET together rather than FO and HO+ET. This is because HO approximates the performance of FO to some degree but not with the same loss of efficiency as using HO+ET over FO+ET. Using the data obtained from just HO and FO+ET offers predictive insights into the performance of HO+ET and FO but at a lower cost than from FO and HO+ET.

6.4 Limitations

The performance models obtained in Chapter 5 have limited predictive power according to their Pearson correlation coefficients. One issue that explains this is that the unit for the symbols were abstract, ignoring the physical differences between the representations of the symbols. As an example, an integer can be treated as being one symbol regardless of its value; however, in reality, the physical representation of an integer in computer hardware has a specific size, so the cost of comparing two large integers might be significantly higher than comparing two small integers. The implication of this is that knowledge of the physical implementation

of generalization is necessary to accurately model its performance.

This relates to one limitation for the NVC measure used in the thesis that each symbol is assumed to be treated as being of a comparable "size" or "weight." Although all atom values in Elixir share a fixed size representation, this is not generally true for other values. One case is that of integers that have a varying length binary representation depending on the size of the integer. In the case of Elixir, this can be treated as an internal implementation detail and can in principle vary depending on the implementation of the virtual machine executing the code.

However, the sizes of integers occurring as literals in source code does not vary to the same extreme as binary literals (such as strings). Because binary literals are treated as constant symbols in this thesis, the commonality from two identical binary values contributes the same to the NVC measure regardless of their size. One approach to account for this would be to represent binary literals as expressions composed of individual byte values. Complexities arise unfortunately for the special case of strings, since the mapping between textual characters and byte values does not necessarily need to be one to one. Dealing with this general problem of differentiating between commonalities at the lower lexical level versus at the higher semantic level does not have a straightforward solution — in fact, this is for what equational theories are needed.

This limitation of the NVC measure is likely a key factor in the limited capability of explaining variance of performance timing using the NVC measure of the LGG. Clearly, a matching operation between two binary values that are a hundred bytes long will involve more CPU cycles than when two values represented by only four bytes are matched. This underestimation of the gain in commonalities (estimated by our NVC measures) affects the effectiveness measure, making it lower if some constant symbols should have a higher weight than our NVC measure gives.

Similarly, the NVC measure for the first-order implementations could potentially have higher values if the encoding of lists were based on a 'cons' structure approach (i.e., [1, 2] corresponding with 'cons(1, cons(2, nil))') which not only has additional 'cons' symbols but allows for generalizations when only the first elements in two lists being generalized are the same. For example, [1, 2] and [1] can be generalized as cons(1, X) if using this approach of encoding list terms instead of treating the terms like list(1, 2) with an arity corresponding with the list length. However, this limitation is not concerning for interpreting our results because all of the implementations would see increased NVC measures and because our analysis focuses on comparing relative differences between the implementations rather than simply absolute measures.

Overall, this highlights the importance of the interplay between the physical implementation of computing versus the purely theoretical concepts. Given a goal of building an accurate performance model of a concrete generalization process we can adjust the logical model (i.e., how to map physical representations and abstract

logical symbols) to better fit the real-world measurements. For example, the symbols we are using can be derived from physical measurements such as timings of compute operations. Alternatively, if the logical model is treated as immutable then the theoretical performance predictions can in principle be obtained by building the physical hardware that corresponds with that model. This could mean, for example, that the physical size of registers in hardware need to be increased to allow faster comparison of larger integers.

6.4.1 Implementation limitations

A few limitations of this work are simply a result of the limited scope of implementation and would require more development work to overcome.

One case is that, after obtaining generalized terms by applying anti-unification to input terms, there can be additional rounds of anti-unification applied to generalized terms. This thesis only studied the first round of anti-unification where the terms being generalized are ground terms. But the results of this thesis should still be applicable to the general problem.

We additionally do not consider anti-unifying terms obtained from instantiated generalized terms which may contain **nil** values introduced due to our equational theory. Such terms could be simplified to equivalent terms without the **nil** values acting as padding. However, this introduces additional complexity when measuring the number of commonalities since these values can either be included or excluded from our NVC measure.

More generally, depending on the equational theory we may be able to rewrite the input terms not just into larger terms but also into smaller terms, thus reducing them. If such simplified terms are used to form a generalized term then the number of commonalities between the inputs will not entirely be known from the generalized term alone. Therefore the quality of the generalization can be entirely independent from the syntactic similarity (and consequently lexical similarity) between the generalized term and the input terms. Because of the specific equational theory studied in this thesis, which only adds or removes **nil** terms, this issue is avoided.

6.5 Threats to validity

Threats to validity include construct validity, internal validity, and external validity.

6.5.1 Construct validity

The effectiveness measure used in this work is derived from a measure of commonalities between terms and a measure of execution time. Therefore the validity of the effectiveness measure depends on the validity of

the two others as well as the validity of this choice.

As explained previously the number of commonalities does not capture a gain from computing an "alignment" between terms when an anti-unification form with equational theory is used. A different formulation of effectiveness would be justified if this kind of gain were of interest. However, in the context of comparing against forms without equational theory, the chosen effectiveness measure is reasonable.

In turn, the measure of cost was chosen as the single-threaded execution time with data transmission costs minimized as much as possible. The cost of memory storage and the transfer between storage and the CPU were not considered. A different formulation of effectiveness incorporating such other forms of cost would be justified if these costs were of interest. However, this significantly increases the scope and complexity of such a study and so it was not feasible to be included in this thesis.

6.5.2 Internal validity

Threats to internal validity include defects in the implementation as well as the effect of taking timing measurements upon performance. While best efforts were made to mitigate the risk of errors in implementing the anti-unification algorithms there is a possibility of some errors not being noticed. The impact of timing measurements on performance is expected to be minimal but is not problematic as there should be no difference in impact between the implementations allowing for a fair comparison.

6.5.3 External validity

The results of the thesis are unlikely to be completely generalizable to different datasets or to different implementations of the algorithms. For example, some of the performance differences between FO and HO are specific to this dataset. Depending on the dataset, it is possible that HO will generally be more effective than FO or that FO will be more effective for every data instance.

The division of a codebase into files is programming language specific. For example, Java has an unusual restriction that a file must correspond one-to-one with a class. Other languages such as Elixir have different conventions for what unit of source code a file should correspond with. Thus, we do not claim that our dataset of Elixir source code would be representative of a generalized set of source code written in other languages.

Different programming languages also have different conventions for what should constitute "source code". In Java for example, documentation can be stored in comments called "document comments" and, since comments in Java are essentially ignored by the compiler, this information would typically not be regarded as constituting source code. Elixir, however, has a convention of storing documentation in module attributes

that the compiler does process and so documentation in Elixir should constitute as source code. For this reason, the performance characteristics of generalizing real-world Elixir source code may differ from generalizing real-world source code in other languages as a consequence of these differing conventions.

6.6 Summary

The results from studying different forms of anti-unification were discussed in this chapter. The three key results obtained were that (a) semantic generalization is more effective than syntactic generalization; (b) semantic generalization is less efficient than syntactic generalization; and, (c) the quantified trade-off between effectiveness and efficiency has some generalizability between different forms.

We found that lower order forms generally outperform higher order ones. Likewise for equational theories, we found that the forms without equational theory overall outperform the forms with equational theory.

Limitations and potential threats to validity for these results were discussed.

Chapter 7

Conclusion

Generalization is a key problem in engineering of software systems because of its importance to analyzing source code. Specifically, generalization is important because it allows for extraction of recurring patterns, structures, and functionalities across different portions of a codebase.

This thesis explored different forms of anti-unification for generalizing Elixir source code, focusing on first-order (FO) versus higher-order (HO) approaches, both with and without equational theory (ET). After establishing the theoretical foundations and formal definitions, a methodology was developed to compare these forms using effectiveness and efficiency measures, particularly non-variable count (NVC) and performance timing. The implementation phase covered source code encoding, algorithm implementation including Needleman–Wunsch, and measurement approaches. Performance modelling revealed that the NVC of the least general generalization (LGG) correlated well with execution time, though with limited predictive power. Linear models better fit FO and HO forms, while power models better fit forms with equational theory. Key findings demonstrated a clear trade-off between effectiveness and efficiency. Forms with equational theory showed significantly higher effectiveness (FO+ET: 590%, HO+ET: 703% compared to FO baseline) but were approximately 1000 times less efficient, with median ratios of 400–500 times slower on individual inputs. While higher-order forms generally showed lower efficiency than first-order forms, they occasionally (in about 5% of cases) demonstrated 30–70% better efficiency. The research supports that semantic generalization (using equational theory) achieves greater effectiveness but at a relatively substantial computational cost compared to syntactic generalization, with quantifiable trade-offs that show some generalizability across different forms. First-order approaches typically outperformed higher-order ones in efficiency, though with notable exceptions.

These results support the thesis of this dissertation that: (a) semantic generalization is more effective than syntactic generalization; (b) semantic generalization is less efficient than syntactic generalization; and, (c) the quantified trade-off between effectiveness and efficiency has some generalizability between different forms.

In the following sections we discuss contributions, shortcomings of sequence alignment, and future work.

7.1 Contributions

This thesis makes several significant contributions to the field of program analysis and source code generalization:

First, we developed and formalized a novel approach to applying anti-unification to Elixir source code, introducing a compact quoted expression representation that supports both first-order and higher-order forms, as well as syntactic and equational theory-based analysis. This representation provides a foundation for future work in source code analysis and transformation in functional programming languages.

Second, we introduced a methodology for quantitatively comparing different forms of anti-unification, introducing precise measures for both effectiveness and efficiency. The non-variable count (NVC) metric (based on the formal concept of least general generalization) and performance timing measurements provide a systematic framework for evaluating generalization approaches that can be applied in future research.

Third, we conducted a detailed empirical analysis of the trade-offs between syntactic and semantic generalization in the context of source code analysis. Our findings quantified the relationship between effectiveness and efficiency, demonstrating that semantic generalization (using our equational theory) achieves 6–7 times greater effectiveness while incurring a computational cost factor of approximately 1000. This analysis provides concrete guidance for practitioners choosing between different generalization approaches based on their specific requirements.

Finally, we developed performance models that, while showing limited predictive power, establish a methodology for analyzing and predicting the computational costs of different anti-unification approaches. These models provide a starting point for future research into performance optimization and prediction in source code generalization systems. These contributions advance our understanding of source code generalization and provide practical tools and insights for both researchers and practitioners working with program analysis and transformation systems.

7.2 Shortcomings of sequence alignment

Our equational theory for block symbols that correspond with sequences of terms (potentially with hierarchical structure) addresses the problem of finding an alignment at the "root" level of the block. This means the hierarchical structure between terms in the sequence and their possible alignments is left unaddressed by using just sequence alignment. More sophisticated methods for finding alignments between hierarchical structures (i.e., tree alignment) are possible and are part of potential future work described in the next section.

Another shortcoming is that the global sequence alignment approach does not consider the number of distinct variables in the generalization formed after alignment. For this reason generalizations that better capture structural patterns might not be produced if relying on just the NVC metric. Unfortunately, the polynomial time tractability of sequence alignment may be lost if we wish to optimize for the number of distinct variables in the final generalization. In other words, the dynamic programming method for solving sequence alignment which provides a polynomial time solution cannot be applied unless the problem decomposes into reusable parts.

7.3 Future work

In this section, we describe some directions for future work, including improved sequence alignment, the possibility of supporting more than two inputs and of leveraging macro expansion to measure semantic similarity.

7.3.1 Supporting more than two inputs

The current implementations only support anti-unification of two terms. However, the theoretical framework is compatible with the generalized problem of anti-unifying any number of terms. One approach to implementing this is to reduce the cases of more than two terms to iterative application of anti-unification on two terms. For example, antiunify(a, b, c) can be implemented by antiunify(antiunify(a, b), c). The issue is that anti-unification is not generally associative, and so an ordering would have to be chosen. Should it be a fixed ordering or based on properties of effectiveness and/or efficiency? This question remains open. We note that Mahmoud et al. (2024) used a heuristic approach to address this question, selecting the most "similar" pairs (according to a specific notion thereof) to anti-unify first; this was aimed primarily at achieving better effectiveness obviously, but also at improving efficiency to some extent since time would not be wasted in anti-unifying dissimilar inputs. We note that that work did not delve thoroughly into discovery of the

optimally effective nor optimally efficient anti-unifier among the large number of possibilities investigated.

7.3.2 Macro expansion

The current implementations ignore the distinction between macros and functions in source code and for these cases we perform only a syntactic generalization. Evaluating a macro results in transforming the input expression into a semantically equivalent expression. This technique is used to implement domain-specific languages (DSLs), enabling source code written in different DSLs to be embedded within Elixir source code and to interoperate with it.

As an example, Listing 7.1 shows the macro expansion of the expression **if true do** 1 **end**. This demonstrates the fact that **if** is treated in Elixir as "syntactic sugar" for a **case** expression.

This means that the semantics of the **if** expression is defined in terms of the **case** expression. So if we wish to perform generalization between an **if** and **case** expression that considers semantic equivalence, then the necessary equational theory for this is actually already defined and implemented (through the macro definition and its expansion).

As another example, Listing 7.2 shows the macro expansion of the expression **true** and 1. Again, and is treated as "syntactic sugar" for a **case** expression revealing a commonality in the semantics between **if** and and which is not evident from a purely syntactic comparison.

When generalizing two expressions that have commonalities in semantics, there is a challenge that rewritten terms may not be syntactically similar to either of the original inputs. This means that the syntactic commonalities found between the rewritten terms do not need to reflect syntactic commonalities found between the original terms. In the examples shown, the syntactic similarity actually decreases after expanding the macros. This raises the issue of how to measure effectiveness in a valid manner that weighs semantic similarity higher than syntactic similarity. Otherwise the expected gain from identifying semantic similarity could instead be unintentionally measured as a decrease in the effectiveness of finding syntactic similarity. One possible approach to address this is to introduce a multi-dimensional measure that avoids reducing the different similarity measures into a single scalar quantity.

```
1 case true do
2   x when x in [false, nil] -> nil
3   _ -> 1
4 end
```

Listing 7.1: Elixir macro expansion of an 'if' expression.

```
case true do
  false -> false
  true -> 1
  other -> :erlang.error({:badbool, :and, other})
end
```

Listing 7.2: Elixir macro expansion of an 'and' expression.

7.3.3 Discovery of generalizations

Another promising direction for future research centres on optimizing the discovery of generalizations through search-based approaches rather than direct computation. Given that computing generalizations — particularly those involving equational theories — can be relatively computationally expensive, there is significant potential in leveraging existing patterns within source code datasets. The key insight is that many useful generalizations may already exist within the codebase itself. Instead of computing these generalizations through expensive anti-unification operations, we could develop efficient search mechanisms to identify existing code patterns that effectively serve as generalizations. This approach could dramatically reduce computational costs while still capturing meaningful patterns in the code.

Rather than computing generalizations through expensive anti-unification operations, we propose studying how existing code patterns propagate through software systems. This shifts the focus from computation to discovery, leading to several concrete research questions:

(1) How frequently do meaningful generalizations already exist within codebases? (2) What are efficient indexing strategies for identifying potential pattern matches? (3) Can we develop heuristics to predict when searching will be more effective than computing new generalizations?

This research direction directly builds on our empirical findings about computational costs. Given that semantic generalization with equational theory is approximately 1000 times less efficient while providing 6–7 times greater effectiveness, finding existing patterns could offer the best of both worlds — the power of semantic understanding without the computational overhead. This reframes our research direction from "how do we compute these patterns?" to "how do we find these patterns when they already exist in the sense of having already been computed?" — a fundamentally different and potentially more tractable problem.

This future direction could significantly impact the practical applicability of source code generalization techniques, particularly in large-scale software systems where computational efficiency is crucial.

Bibliography

Qurat Ul Ain, Wasi Haider Butt, Muhammad Waseem Anwar, Farooque Azam, and Bilal Maqbool. 2019. A Systematic Review on Code Clone Detection. *IEEE Access* 7 (2019), 86121–86144. https://doi.org/10.1109/ACCESS.2019.2918202

María Alpuente, Santiago Escobar, Javier Espert, and José Meseguer. 2014. A modular order-sorted equational generalization algorithm. *Information and Computation* 235 (2014), 98–136. https://doi.org/10.1016/j.ic.2014.01.006 Special issue on Functional and (Constraint) Logic Programming.

Alexander Baumgartner, Temur Kutsia, Jordi Levy, and Mateu Villaret. 2017. Higher-Order Pattern Anti-Unification in Linear Time. *Journal of Automated Reasoning* 58, 2 (01 Feb. 2017), 293–310. https://doi.org/10.1007/s10817-016-9383-3

Peter E. Bulychev. 2009. An evaluation of duplicate code detection using anti-unification. In *Proceedings of the 3rd International Workshop on Software Clones*. 22–27. iESE Report 038.09/E.

Peter E. Bulychev, Egor V. Kostylev, and Vladimir A. Zakharov. 2010. Anti-unification Algorithms and Their Applications in Program Analysis. In *Perspectives of Systems Informatics*, Amir Pnueli, Irina Virbitskaite, and Andrei Voronkov (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 413–423.

Hans-Jürgen Bürckert, Alexander Herold, and Manfred Schmidt-Schauß. 1987. On Equational Theories, Unification and Decidability. In *Proceedings of the 2nd International Conference on Rewriting Techniques and Applications (RTA '87)*. Springer-Verlag, Berlin, Heidelberg, 204–215.

Hans-Jürgen Bürckert, Alexander Herold, and Manfred Schmidt-Schauss. 1989. On Equational Theories, Unification, and (Un)Decidability. *J. Symb. Comput.* 8, 1–2 (July 1989), 3–49. https://doi.org/10.1016/S0747-7171(89)80021-5

Jochen Burghardt and Birgit Heinz. 2014. Implementing Anti-Unification Modulo Equational Theory. *CoRR* abs/1404.0953 (2014). arXiv:1404.0953 http://arxiv.org/abs/1404.0953

Rylan Cottrell. 2008. *Semi-automating small-scale source code reuse via structural correspondence*. Master's thesis. University of Calgary. http://hdl.handle.net/1880/103249

Rylan Cottrell, Robert J. Walker, and Jörg Denzinger. 2008. Semi-automating small-scale source code reuse via structural correspondence. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Atlanta, Georgia) *(SIGSOFT '08/FSE-16)*. Association for Computing Machinery, New York, NY, USA, 214–225. https://doi.org/10.1145/1453101.1453130

Nachum Dershowitz and Jean-Pierre Jouannaud. 1990. Rewrite Systems. In *Formal Models and Semantics*, Jan van Leeuwen (Ed.). Elsevier, Amsterdam, Chapter 6, 243–320. https://doi.org/10.1016/B978-0-444-88074-1.50011-1

Craig Dilworth. 1988. Identity, Equality and Equivalence. *Dialectica* 42, 2 (1988), 83–92. http://www.jstor.org/stable/42969123

Jaroslav Fowkes and Charles Sutton. 2016. Parameter-free probabilistic API mining across GitHub. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Seattle, WA, USA) *(FSE 2016)*. Association for Computing Machinery, New York, NY, USA, 254–265. https://doi.org/10.1145/2950290.2950319

Mark Gabel, Lingxiao Jiang, and Zhendong Su. 2008. Scalable detection of semantic clones. In *Proceedings of the 30th International Conference on Software Engineering* (Leipzig, Germany) *(ICSE '08)*. Association for Computing Machinery, New York, NY, USA, 321–330. https://doi.org/10.1145/1368088.1368132

P. T. Geach. 1951. Frege's *Grundlagen*. *The Philosophical Review* 60, 4 (1951), 535–544. http://www.jstor.org/stable/2181424

Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. https://doi.org/10.48550/ARXIV.1909.09436

Miryung Kim. 2007. Understanding and Aiding Code Evolution by Inferring Change Patterns. In *29th International Conference on Software Engineering (ICSE'07 Companion)*. 101–102. https://doi.org/10.1109/ICSECOMPANION.2007.81

Ulf Krumnack, Angela Schwering, Helmar Gust, and Kai-Uwe Kühnberger. 2007. Restricted Higher-Order Anti-Unification for Analogy Making. In *Proceedings of the 20th Australian Joint Conference on Advances in Artificial Intelligence* (Gold Coast, Australia) *(AI'07)*. Springer-Verlag, Berlin, Heidelberg, 273–282.

J. L. Lassez, M. J. Maher, and K. Marriott. 1988. Unification revisited. In *Foundations of Logic and Functional Programming*, Mauro Boscarol, Luigia Carlucci Aiello, and Giorgio Levi (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 67–113.

Huiqing Li and Simon Thompson. 2010. Similar Code Detection and Elimination for Erlang Programs. In *Practical Aspects of Declarative Languages*, Manuel Carro and Ricardo Peña (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 104–118.

Barbara Liskov and John Guttag. 2000. *Program development in Java*. Addison Wesley, Boston, MA.

May Mahmoud, Robert J. Walker, and Jörg Denzinger. 2024. API usage templates via structural generalization. *Journal of Systems and Software* 210 (2024), 111974. https://doi.org/10.1016/j.jss.2024.111974

Morgan P. Miles, Gregory R. Russell, and Danny R. Arnold. 1995. The quality orientation: An emerging business philosophy. *Review of Business, Saint Johns University* 17 (1995), 7–15.

Webb Miller. 2001. Comparison of genomic DNA sequences: solved and unsolved problems. *Bioinformatics* 17, 5 (2001), 391–397.

Saul B. Needleman and Christian D. Wunsch. 1970. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology* 48, 3 (1970), 443–453. https://doi.org/10.1016/0022-2836(70)90057-4

Catuscia Palamidessi. 1990. Algebraic properties of idempotent substitutions. In *Automata, Languages and Programming*, Michael S. Paterson (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 386–399.

Charles Sanders Peirce. 1902. Logic as Semiotic: The Theory of Signs. In *Philosophical Writings*, Charles Sanders Peirce (Ed.). Dover Publications.

Gordon D. Plotkin. 1970. A note on inductive generalization. *Machine Intelligence* 5, 1 (1970), 153–163.

John R. Searle. 2002. *Consciousness and Language*. Cambridge University Press.

Narges Zirakchianzadeh. 2016. *Automatically Characterizing Logging Usage: An Application of Anti-Unification*. Master's thesis. University of Calgary. http://hdl.handle.net/11023/3524